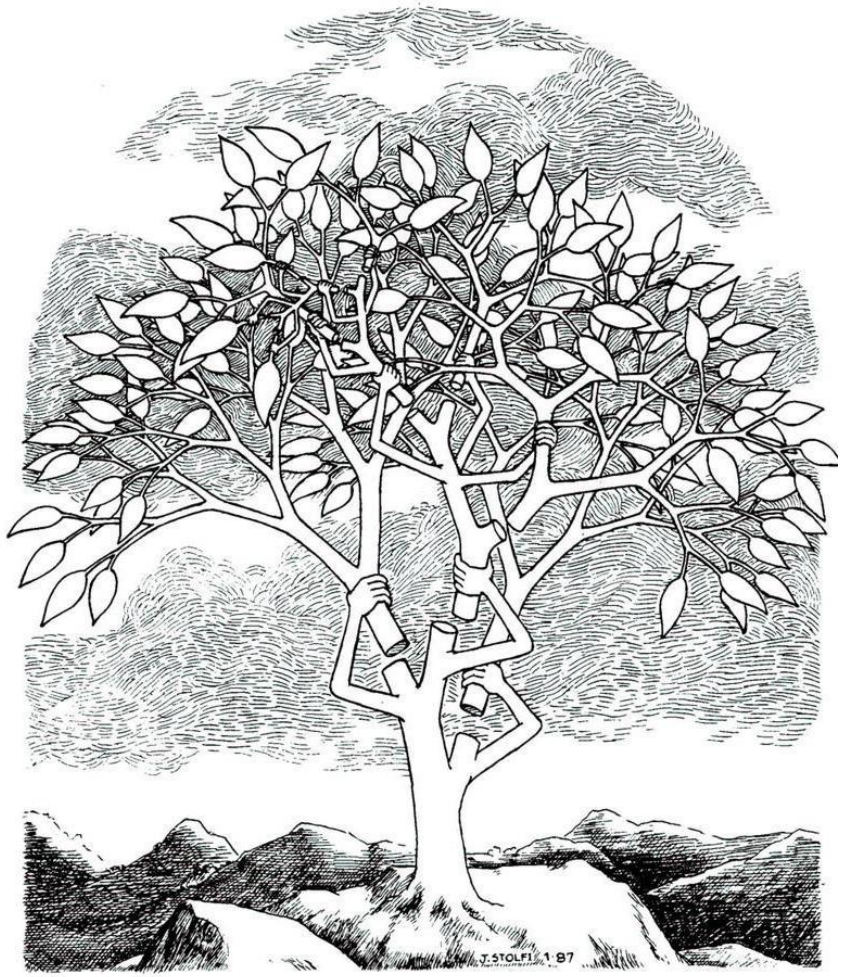


Árvores AVL

Árvore Binária de Busca

Árvore autoajustável

Árvore balanceada

Árvores Balanceadas

Uma árvore binária de busca é dita balanceada se ela possui altura

$$h = O(\log n)$$



n **– número de nós na árvore**

Árvores Balanceadas

✓ **Idéia:** **Manter o custo de acesso na mesma ordem de grandeza de uma árvore binária de busca ótima**

$O(\log n)$

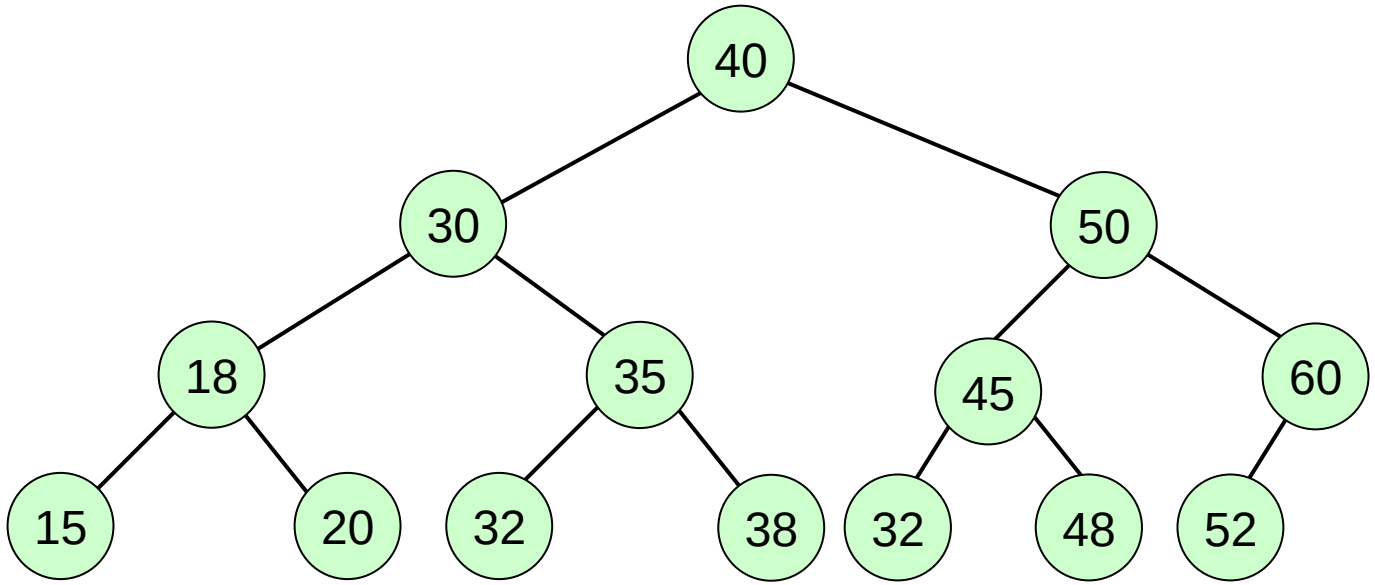


Árvore autoajustável

✓ **Característica:** **A estrutura é alterada periodicamente.**

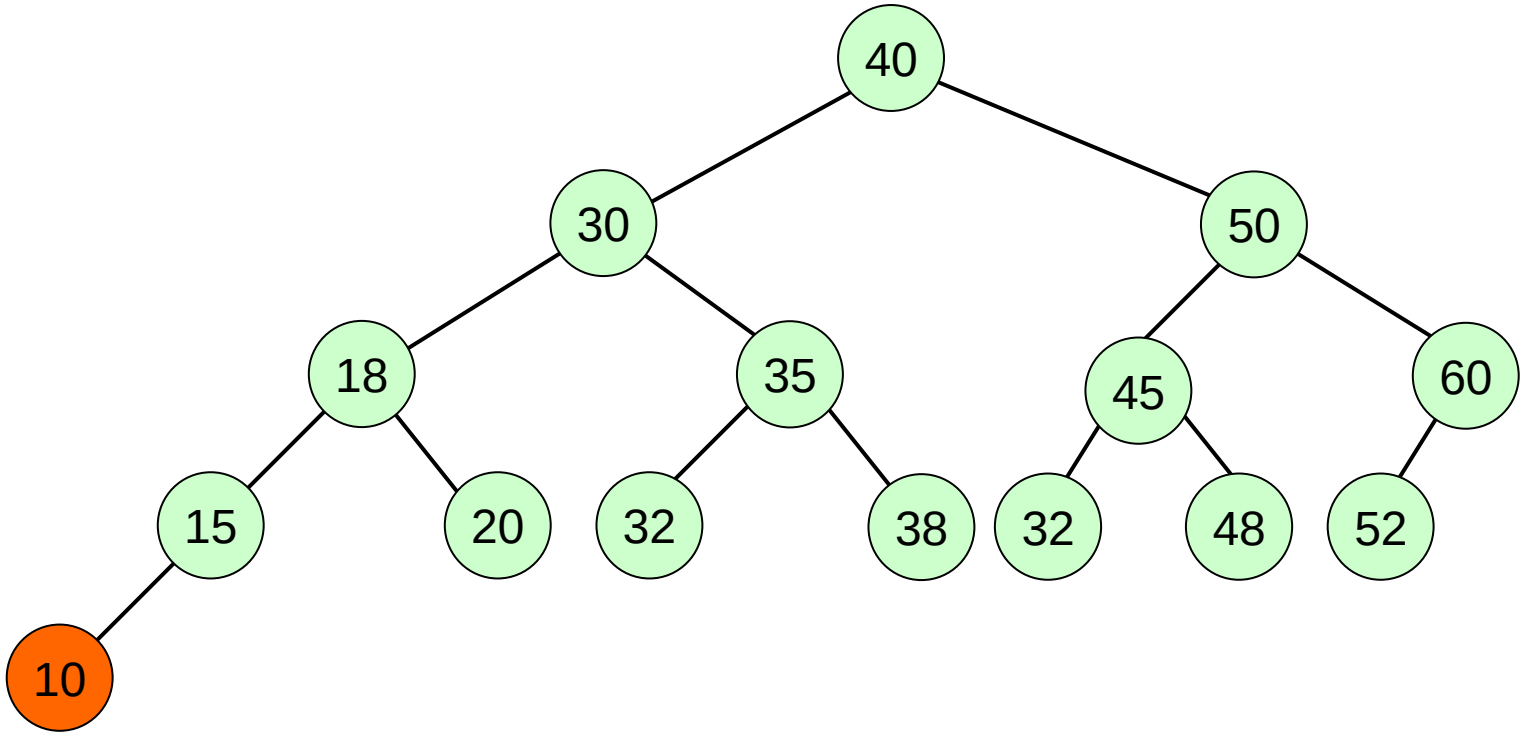
Árvores Balanceadas

Esta árvore é completa, portanto tem altura mínima.



Inserir a chave 10

Árvores Balanceadas



✓ **Para a árvore voltar a ser completa são necessárias Ω(*n*) operações**

Árvores Balanceadas



- ✓ Árvores AVL
- ✓ Árvores Graduadas e Rubro-negras
- ✓ Árvores B

Árvores AVL

Uma árvore binária de busca onde para cada nó v, tem-se

$$|h_E(v) - h_D(v)| \leq 1$$

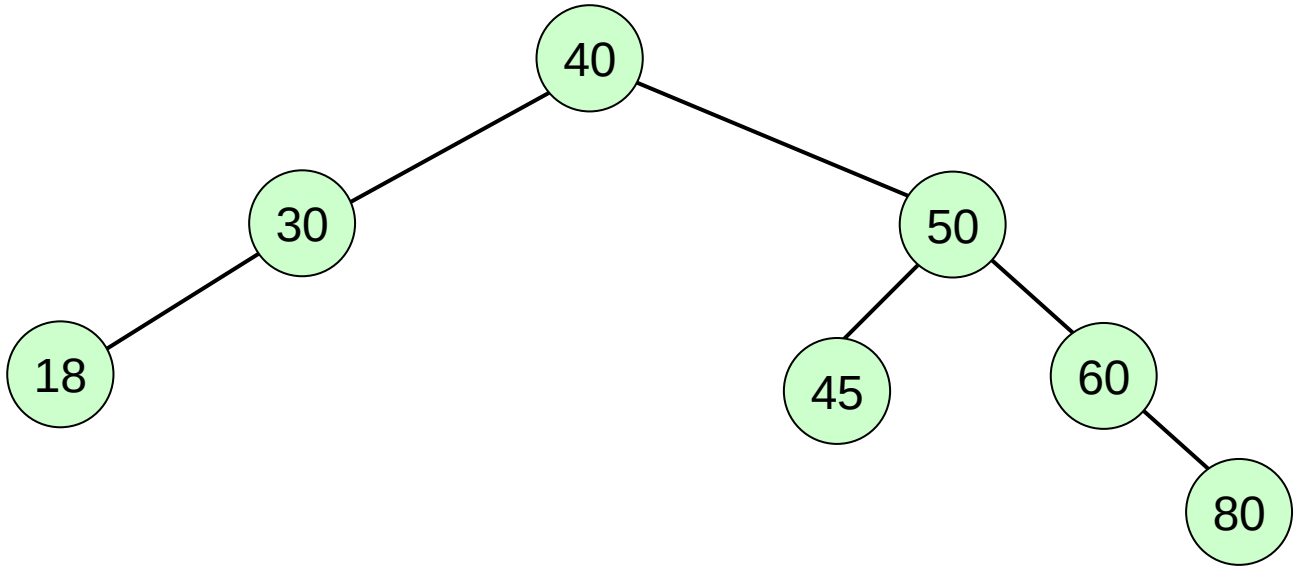
$h_E(v)$ – altura da subárvore esquerda de *v*
$h_D(v)$ – altura da subárvore direita de *v*

Adel'son-Vel'skii e Landis

G.M. Adel'son-Vel'skii, E. M. Landis (1962). An algorithm for the organization of information, *Soviet Mathematics Doklady* 3, 1259-1263.

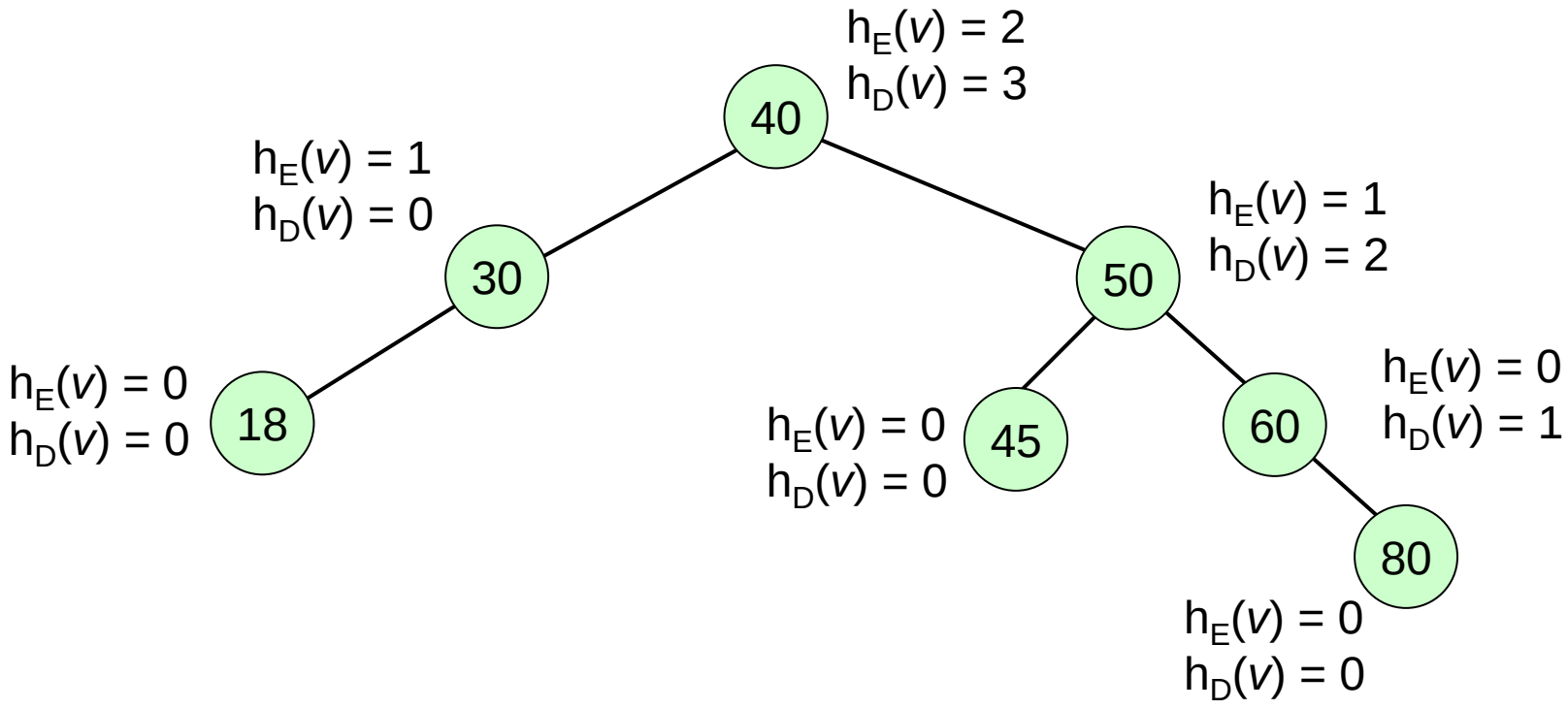
Árvores AVL

Exemplo:



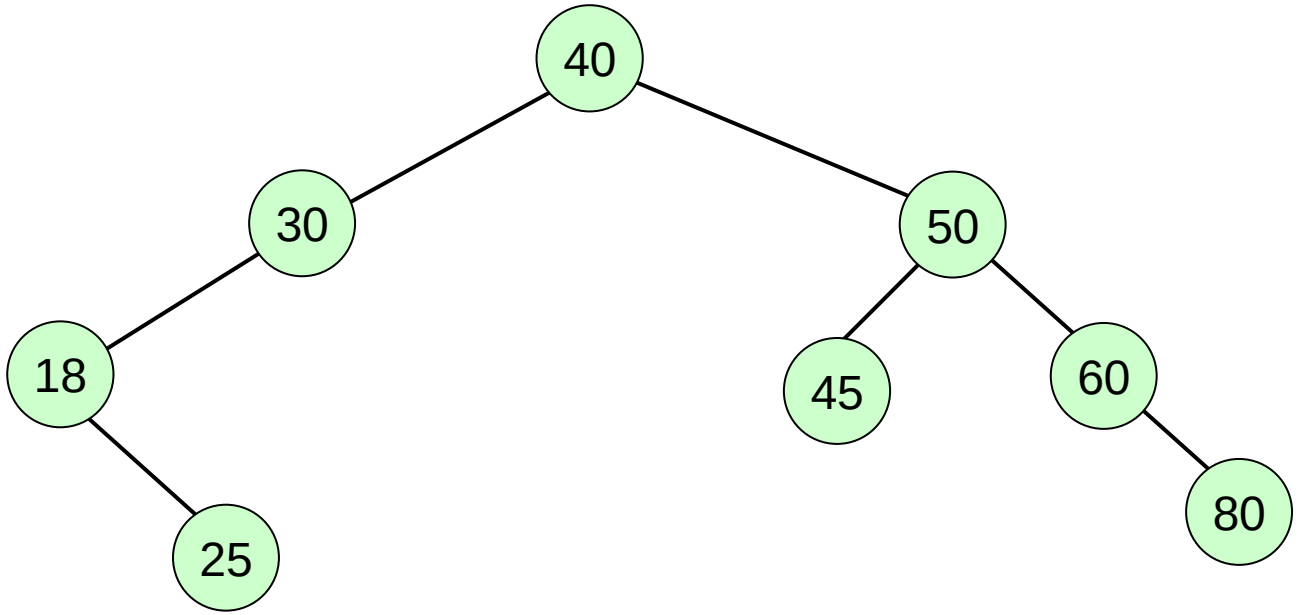
Árvores AVL

Exemplo:



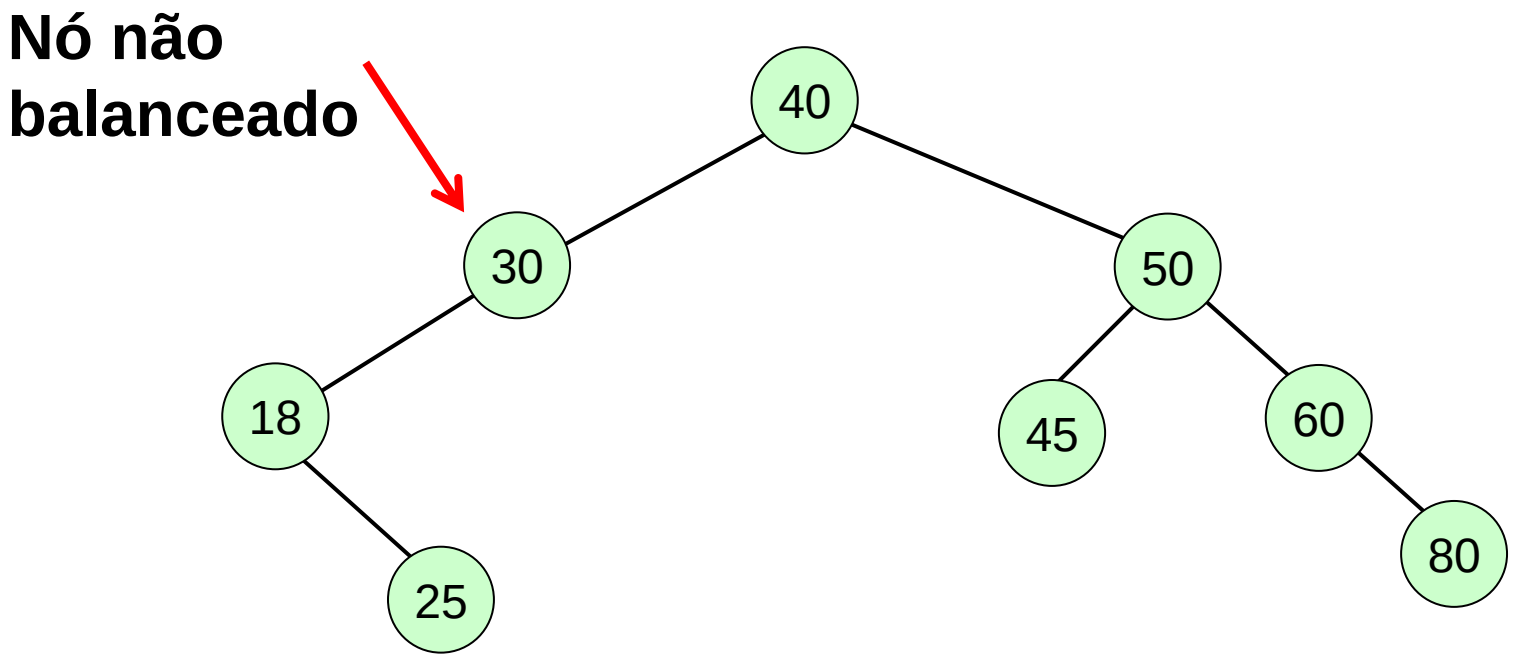
Árvores AVL

Esta árvore é AVL?



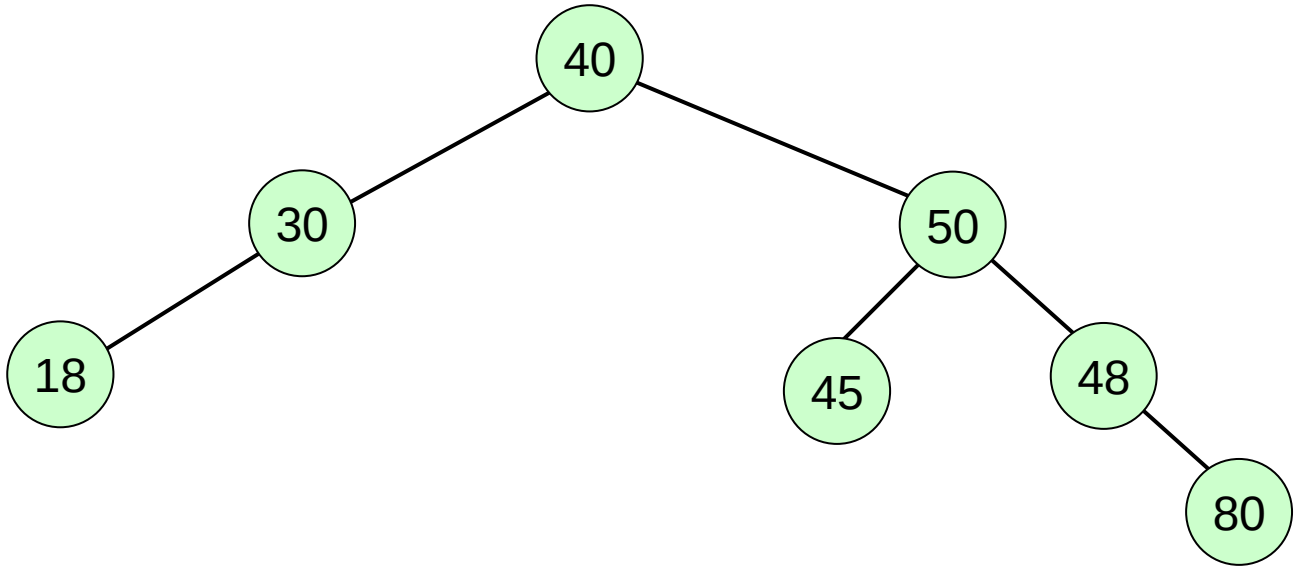
Árvores AVL

Esta árvore é AVL? NÃO



Árvores AVL

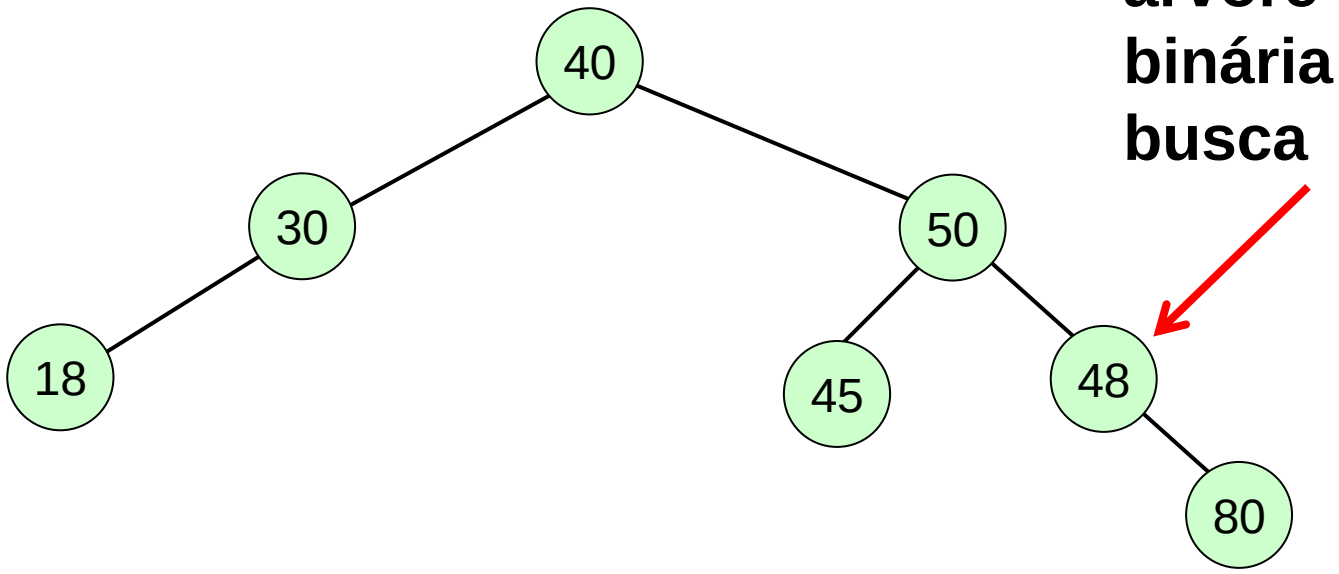
Esta árvore é AVL?



Árvores AVL

Esta árvore é AVL? NÃO

Não é árvore binária de busca



Árvores AVL

Teorema: **Toda árvore AVL é balanceada**

Árvores AVL

Teorema: **Toda árvore AVL é balanceada**

Prova:

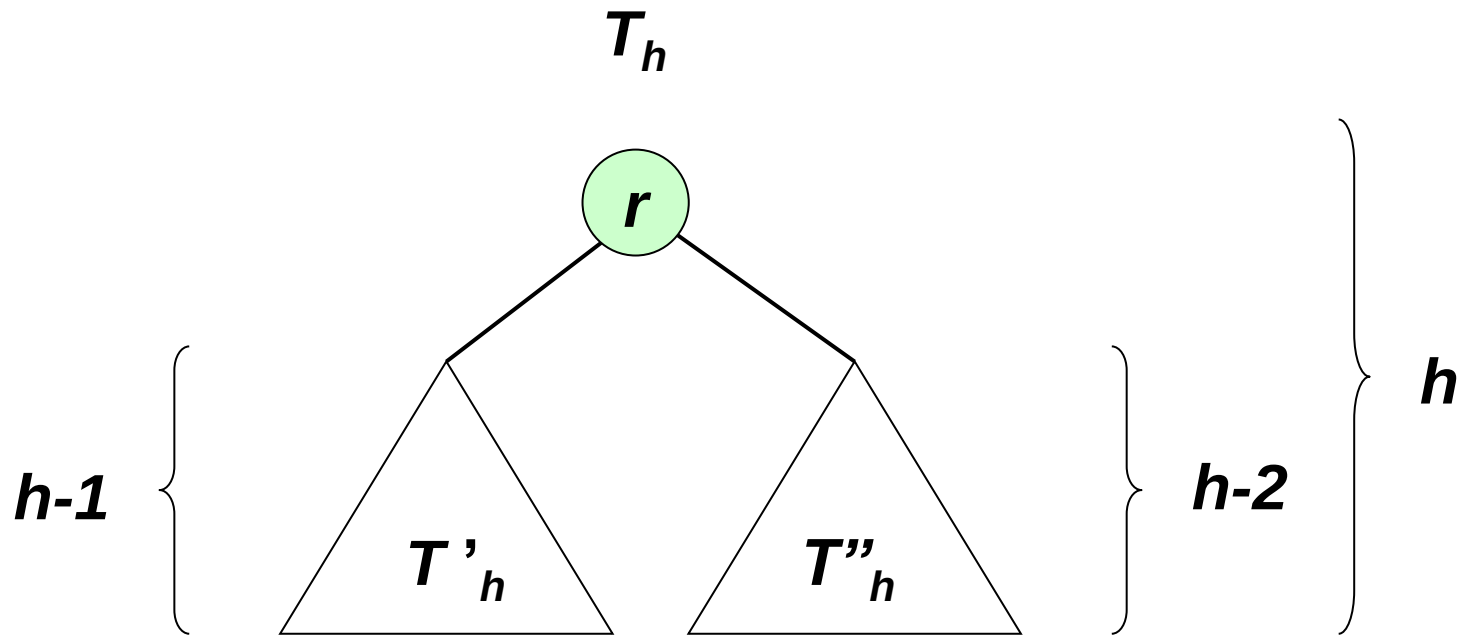
Deve-se mostrar que para um *n* fixo, a árvore AVL de altura máxima possui *h* = *O*(log *n*).

Supor T_h – Árvore de altura *h* com *n* mínimo.

Árvores AVL

Toda árvore AVL é balanceada

Prova:

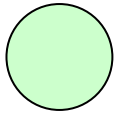


Árvores AVL

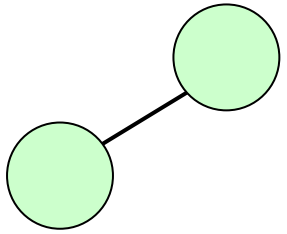
Toda árvore AVL é balanceada

Prova: Recursivamente, temos as seguintes T_h

***h* = 1** **T_1** →



***h* = 2** **T_2** →

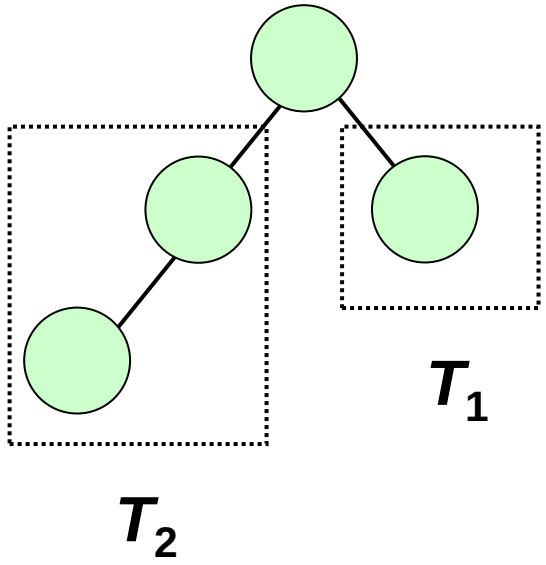


Árvores AVL

Toda árvore AVL é balanceada

$h = 3$

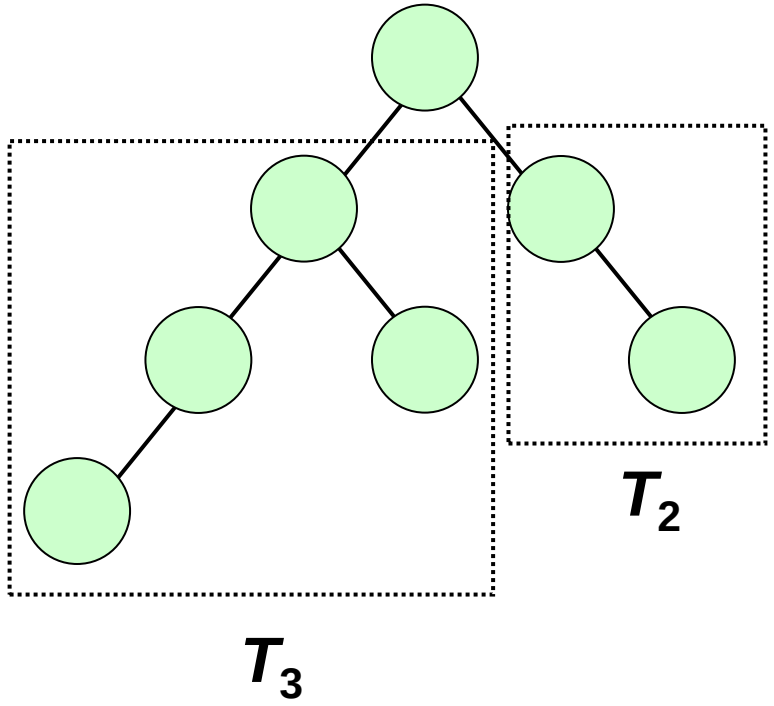
T_3 →



Árvores AVL

Toda árvore AVL é balanceada

$h = 4$ T_4 →



Árvores AVL

Toda árvore AVL é balanceada

Se $h = 0$, então $|T_0| = 0$

Se $h = 1$, então $|T_1| = 1$

Se $h > 1$, então $|T_h| = 1 + |T_{h-1}| + |T_{h-2}|$

Árvores AVL

Toda árvore AVL é balanceada

Se $h = 0$, então $|T_0| = 0$

Se $h = 1$, então $|T_1| = 1$



Se $h > 1$, então $|T_h| = 1 + |T_{h-1}| + |T_{h-2}|$

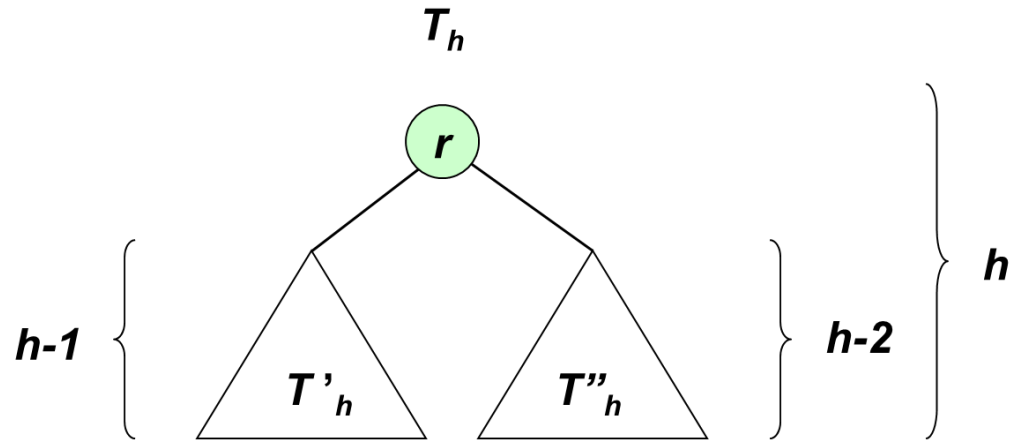
Árvores AVL

Toda árvore AVL é balanceada

Se $h = 0$, então $|T_0| = 0$

Se $h = 1$, então $|T_1| = 1$

Se $h > 1$, então $|T_h| = 1 + |T_{h-1}| + |T_{h-2}|$



Árvores AVL

Toda árvore AVL é balanceada

Se $h = 0$, então $|T_0| = 0$

Se $h = 1$, então $|T_1| = 1$

Se $h > 1$, então $|T_h| = 1 + |T_{h-1}| + |T_{h-2}|$

Como na sequência de Fibonacci

Não tem na sequência de Fibonacci

***h* é *O*(log *n*)**

Árvores AVL

Para *h* > 1, o *h*-ésimo termo da sequência de Fibonacci é:

$$F_h = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^h + \left(\frac{1-\sqrt{5}}{2}\right)^h\right]$$

Como *h* > 0, o termo:

$$\frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^h < 1$$

Árvores AVL

Portanto,

$$|T_h| < \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^h\right]-1$$

Fazendo

$$a = \left(\frac{1+\sqrt{5}}{2}\right)$$

Tem-se

$$\log_a\left(|T_h|+1\right) > h - \log_a \sqrt{5}$$

Árvores AVL

Transformando o logaritmo para a base 2,

$$h < \frac{1}{\log_2 a} \log_2 \left(|T_h| + 1 \right) + \log_a \sqrt{5}$$

Como $|T_h| = n$, h é $O(\log n)$.

***A árvore AVL é balanceada*.**

Árvores AVL

Operações:

Busca – idem árvore binária de busca



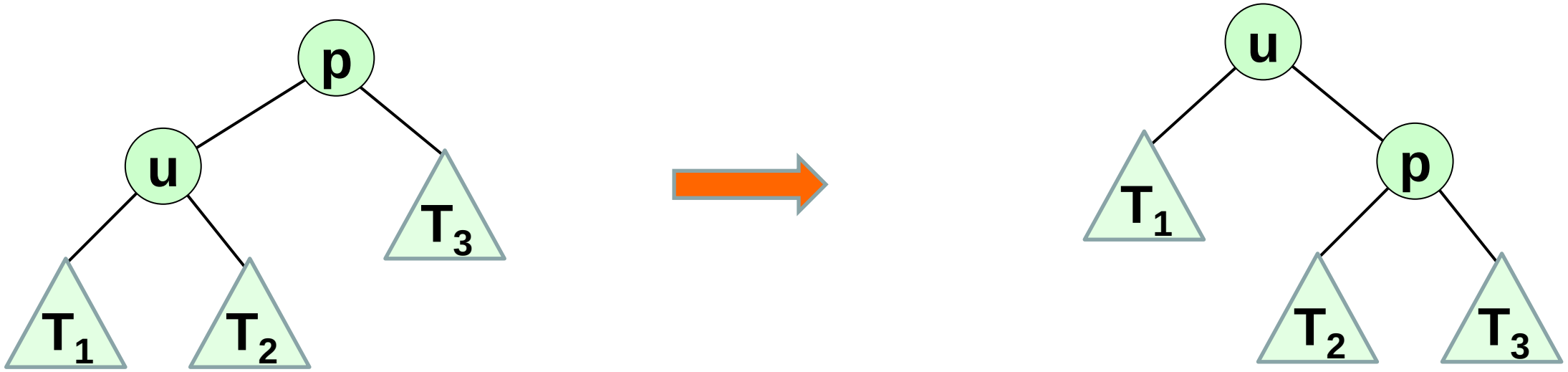
Inclusão



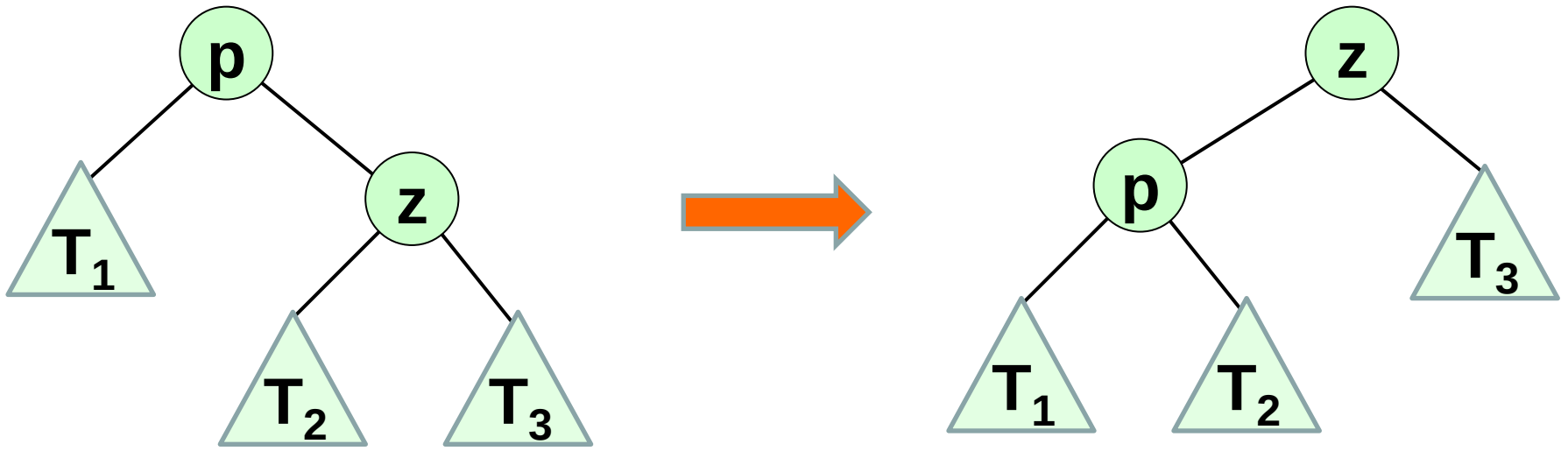
Remoção



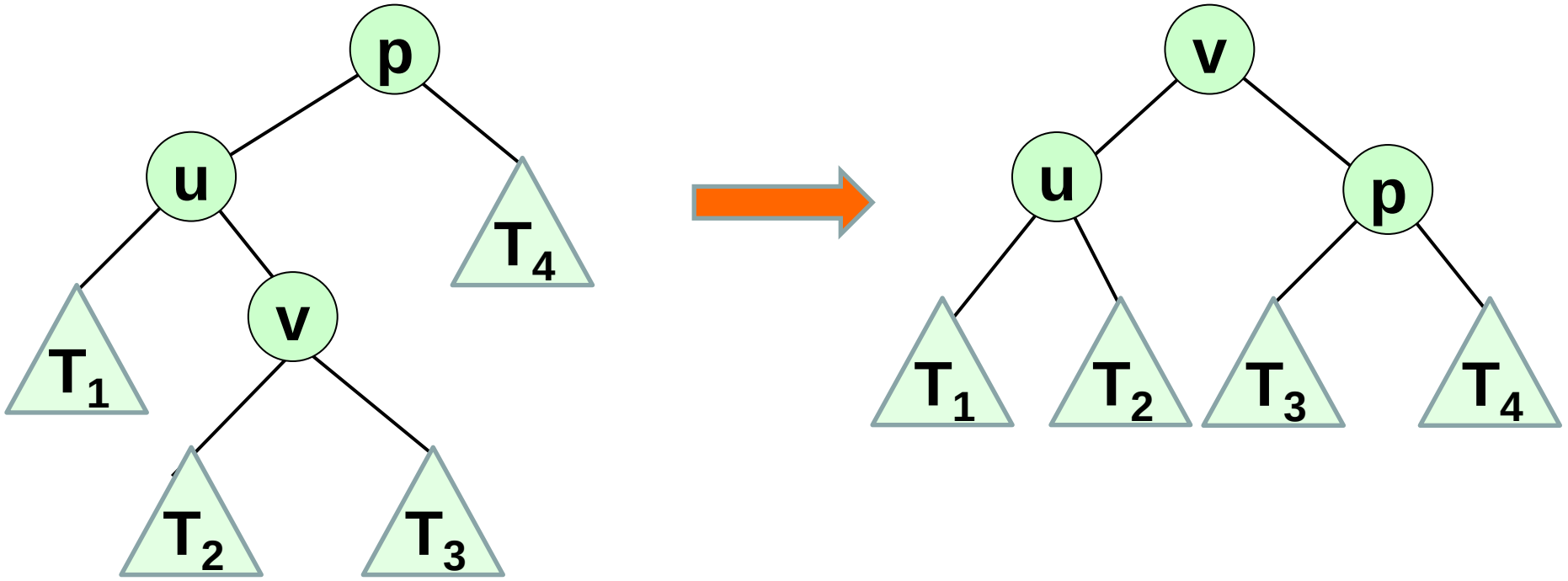
Rotação Direita com raiz p



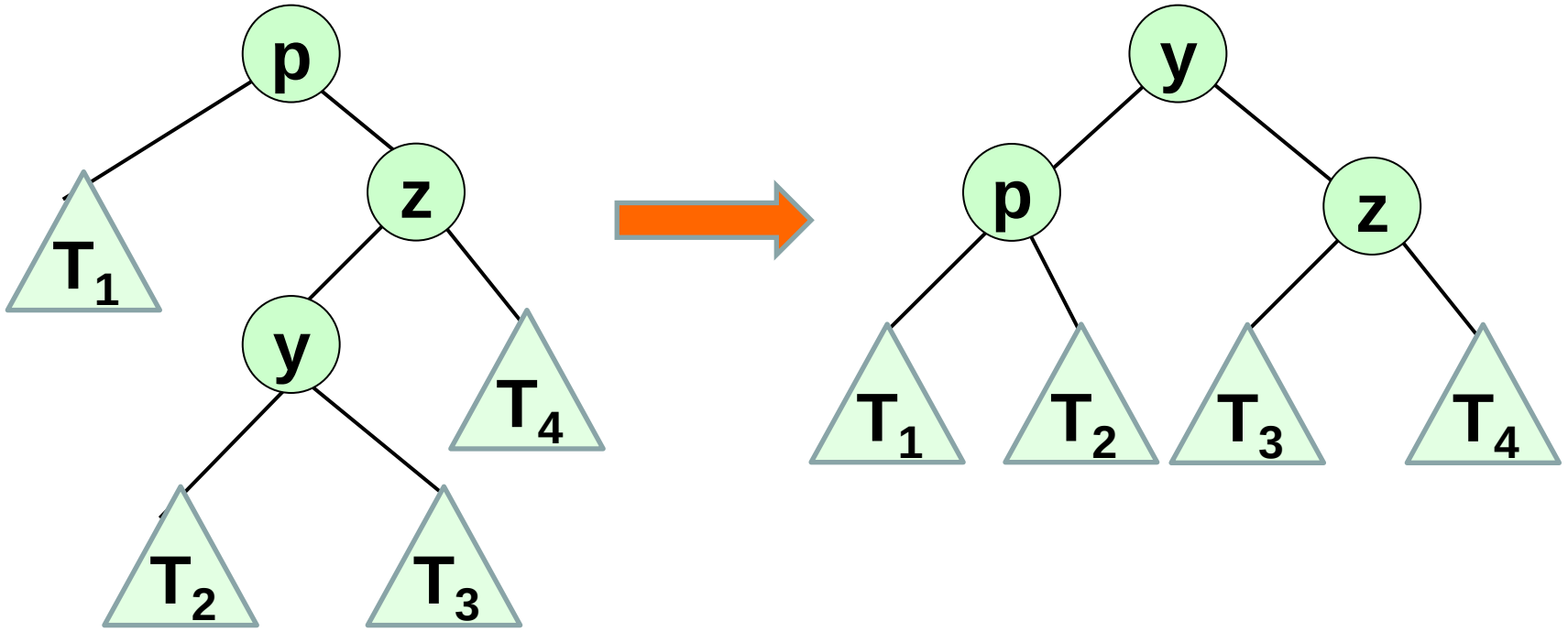
Rotação Esquerda com raiz p



Rotação Dupla Direita com raiz p



Rotação Dupla Esquerda com raiz p



Rotação

O que vocês observaram em relação à altura da árvore antes e depois da rotação?



Exercício

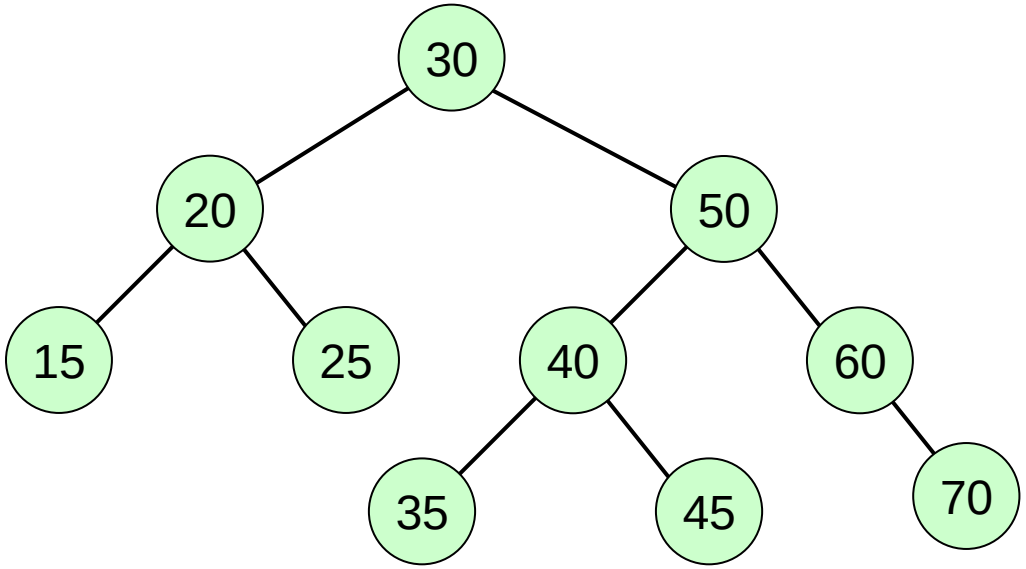
Faça o código das rotações.



Inclusão em Árvore AVL

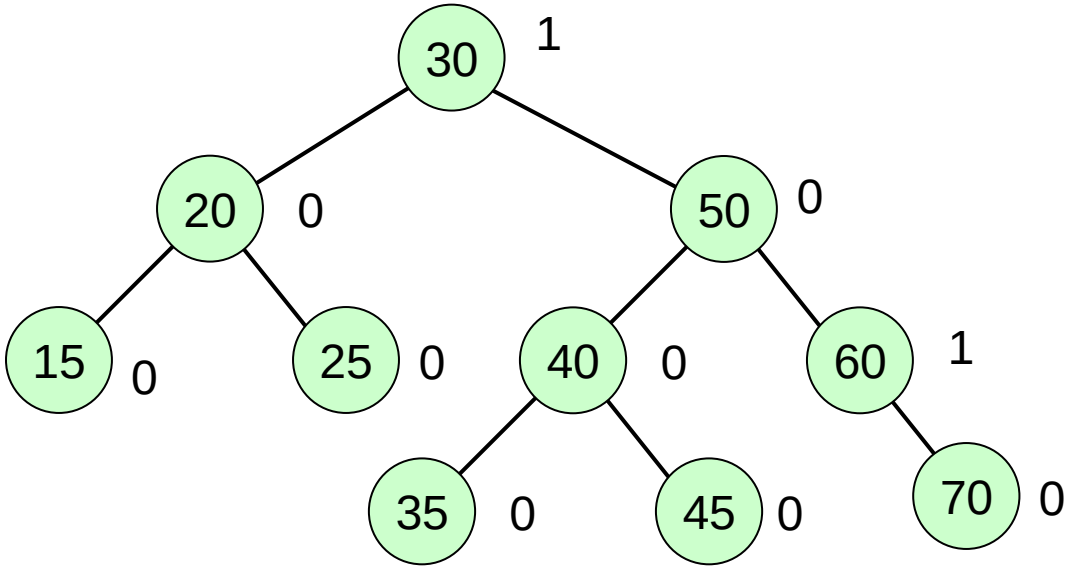
Inclusão em Árvore AVL

Essa árvore é AVL?



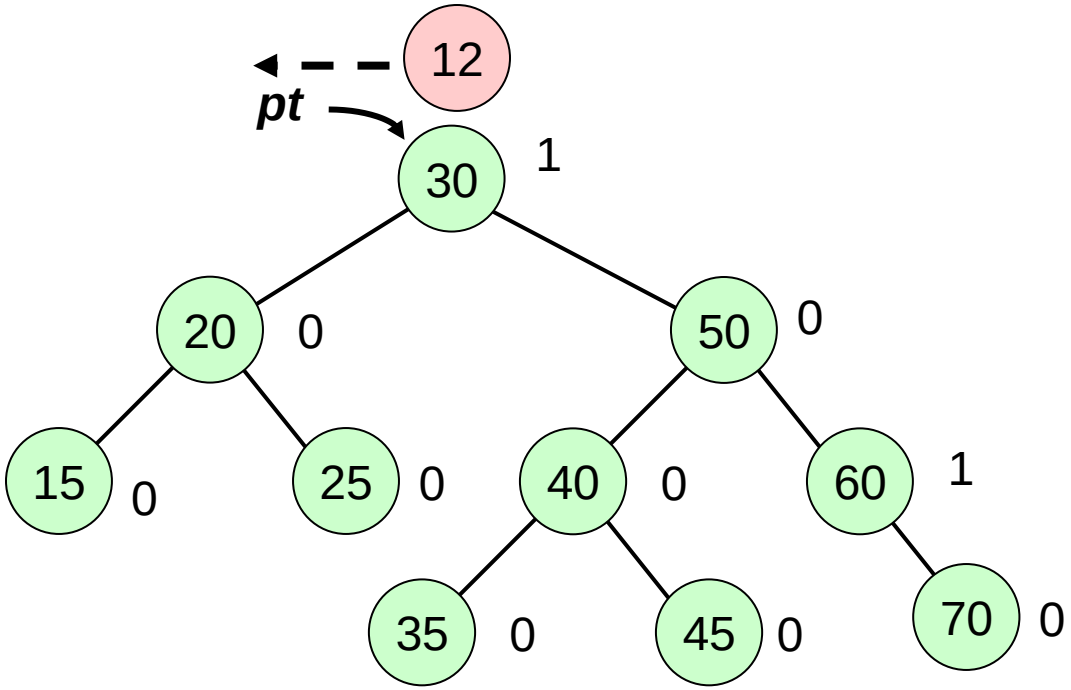
Inclusão em Árvore AVL

Árvore AVL



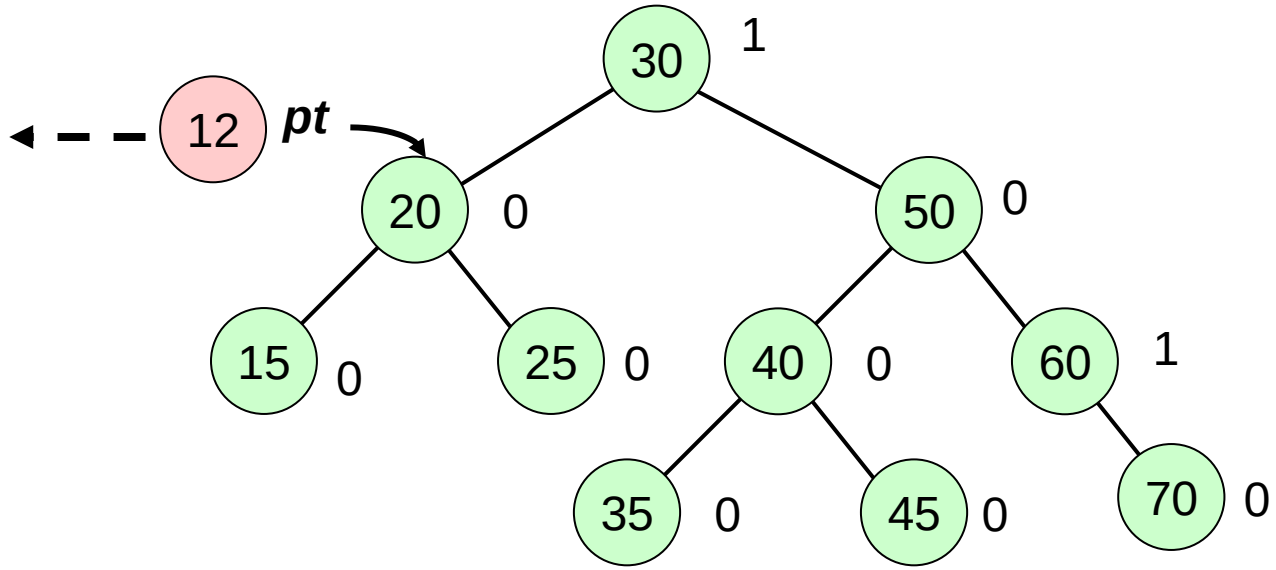
Inclusão em Árvore AVL

Inclusão da chave 12



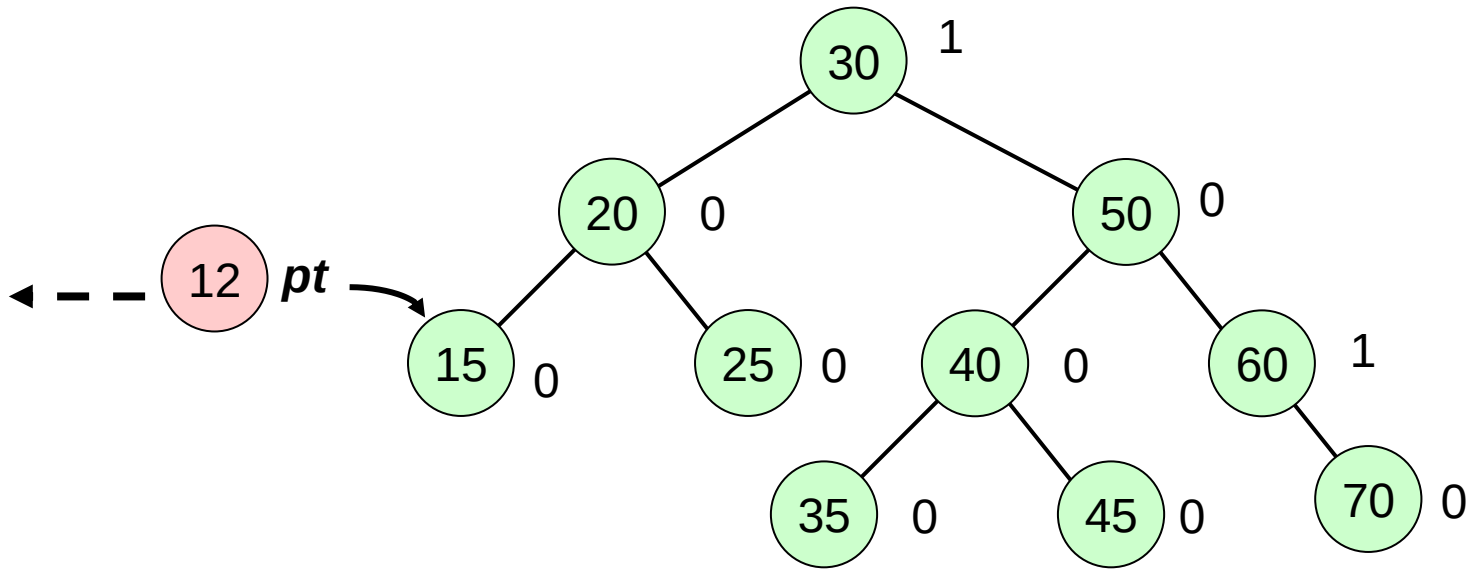
Inclusão em Árvore AVL

Inclusão da chave 12



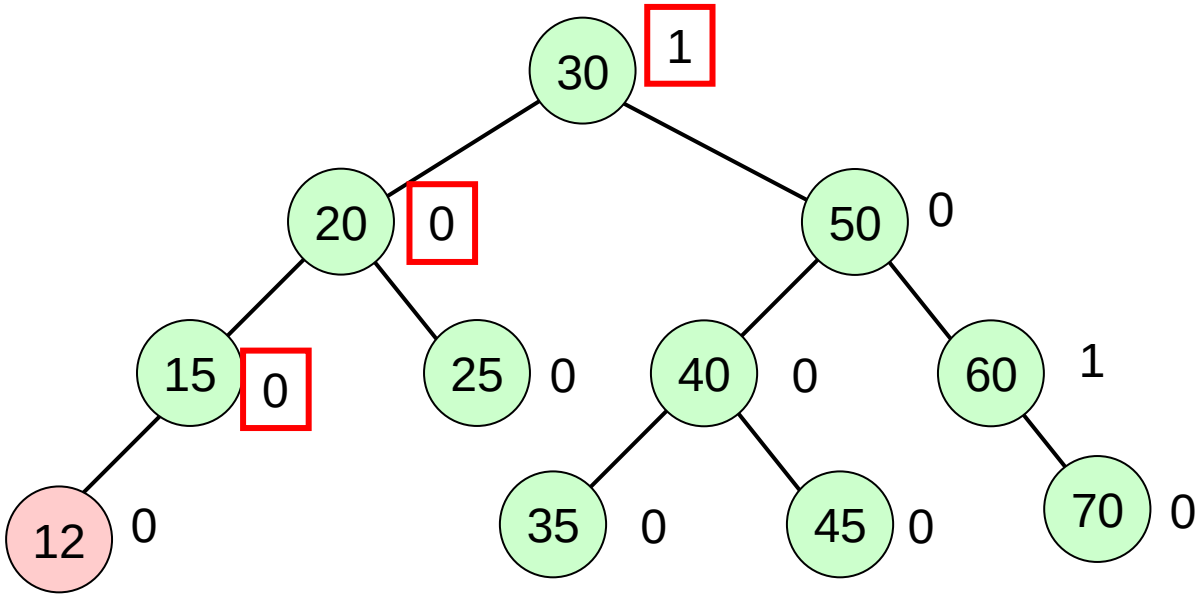
Inclusão em Árvore AVL

Inclusão da chave 12



Inclusão em Árvore AVL

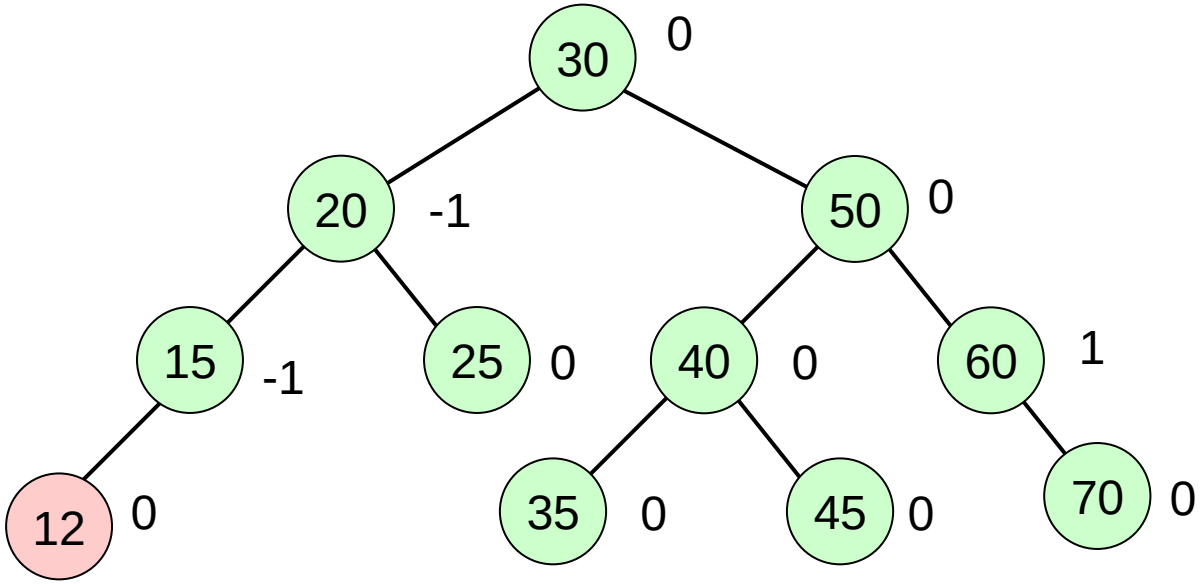
Inclusão da chave 12



O balanceamento pode mudar com a inserção

Inclusão em Árvore AVL

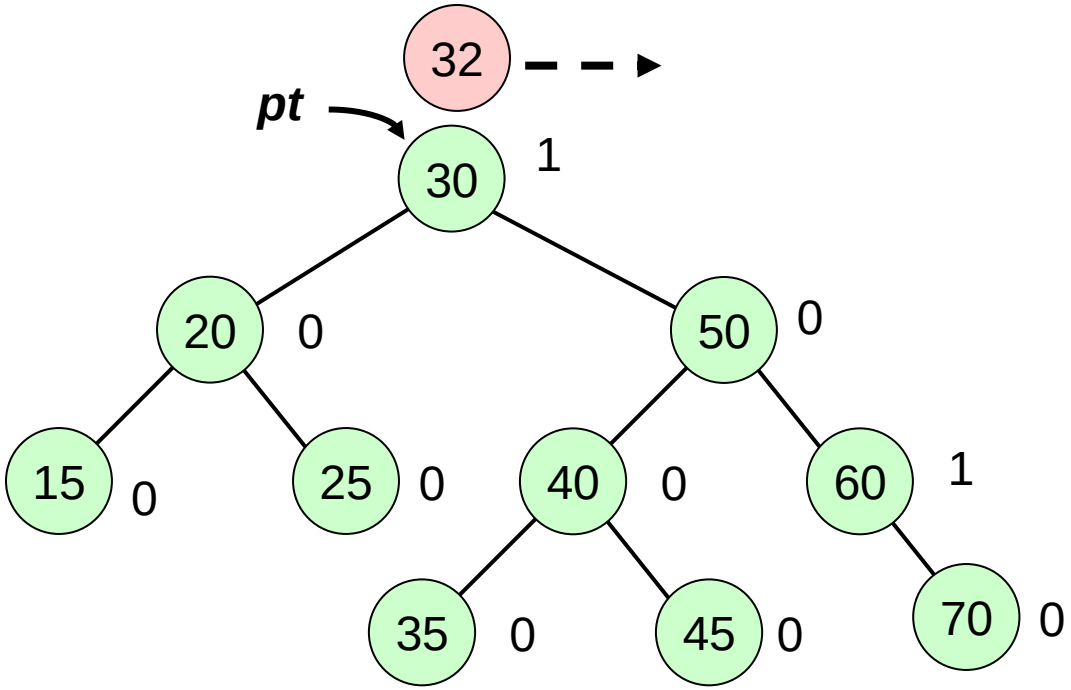
Inclusão da chave 12



OK

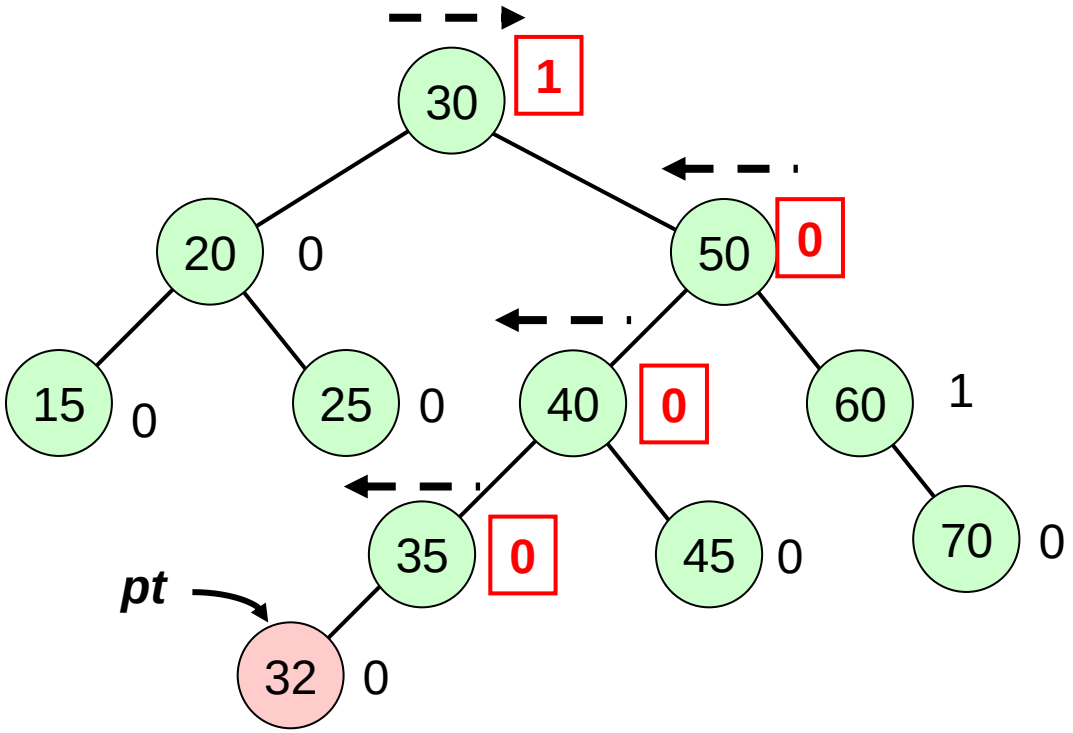
Inclusão em Árvore AVL

Inclusão da chave 32



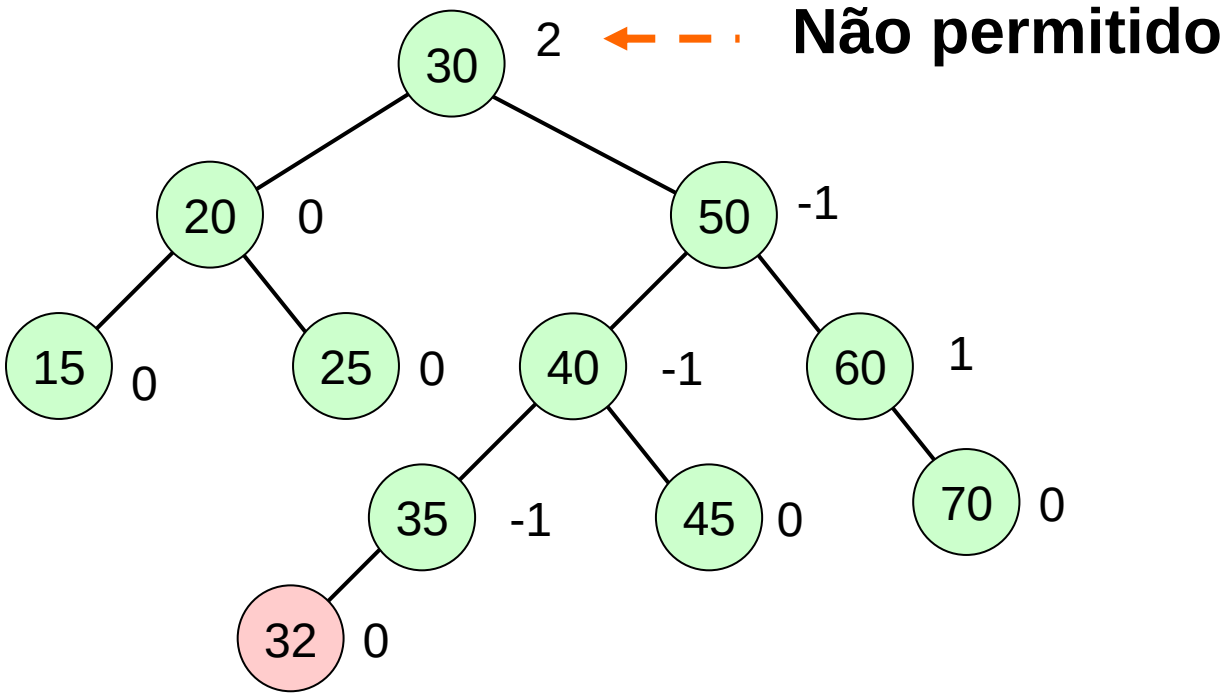
Inclusão em Árvore AVL

Inclusão da chave 32



Inclusão em Árvore AVL

Inclusão da chave 32



Inclusão em Árvore AVL

Como resolver isto?



Inclusão em Árvore AVL

1. Busca chave

2. Se a chave não é encontrada, inclui nó *q*.

3. Se algum nó *p* ficou desregulado,então regular *p*.

OBS:
Quando para algum nó *p*, $|h_E(p) - h_D(p)| = 2$,
É necessário balanceamento.

Inclusão em Árvores AVL

$$\textbf{Balanço}(v) = h_D(v) - h_E(v)$$

Nó regulado: $-1 \leq \text{balanço}(v) \leq 1$

Inclusão em Árvores AVL

Quando $|h_E(p) - h_D(p)| = 2$, pode-se identificar as seguintes situações:

Caso 1. $h_E(p) > h_D(p)$

Caso 2. $h_E(p) < h_D(p)$

Inclusão em Árvores AVL

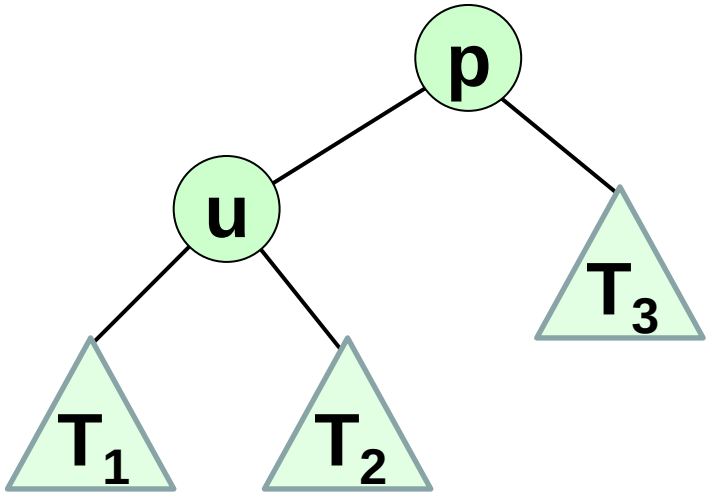
Quando $|h_E(p) - h_D(p)| = 2$, pode-se identificar as seguintes situações:

Caso 1. $h_E(p) > h_D(p)$

Neste caso q pertence à subárvore esquerda de p.

Além disso, p possui filho esquerdo $u \neq q$.

Sabe-se que $h_E(u) \neq h_D(u)$.

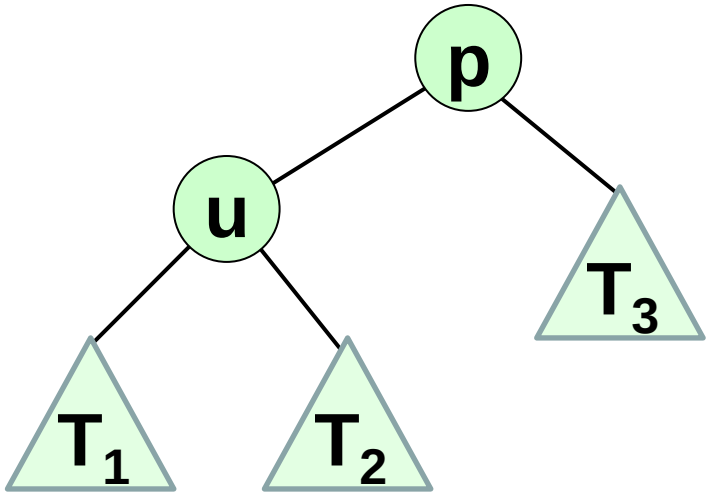


Inclusão em Árvores AVL

Caso 1. $h_E(p) > h_D(p)$

Caso 1.1 $h_E(u) > h_D(u)$

Caso 1.2 $h_E(u) < h_D(u)$



Inclusão em Árvores AVL

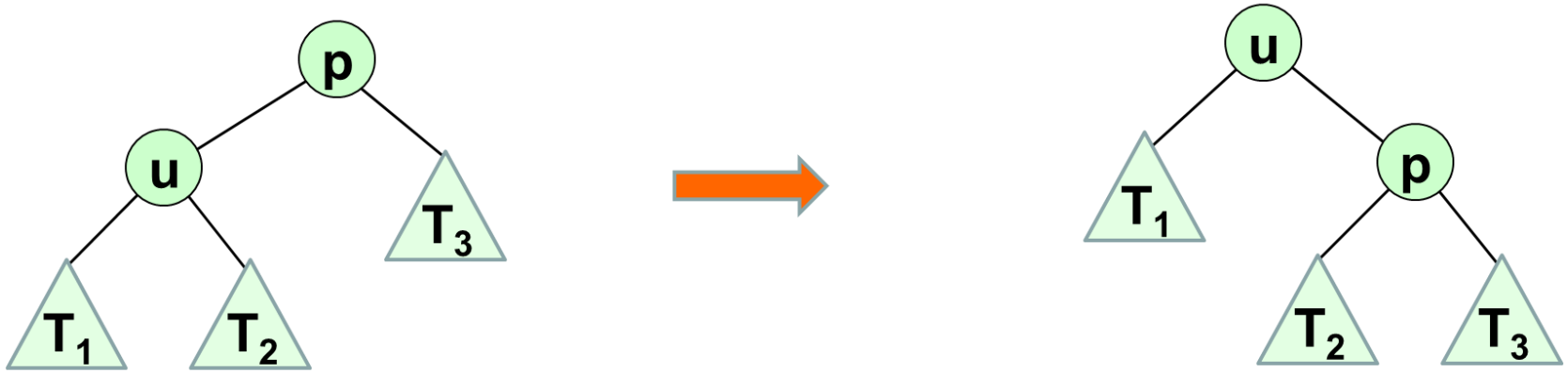
Caso 1.1 $h_E(u) > h_D(u)$

$h(T_1) - h(T_2) = 1$
$h(T_2) = h(T_3)$

Aplicar Rotação Direita

$|h_E(p) - h_D(p)| = 2$
Caso 1. $h_E(p) > h_D(p)$

Neste caso q pertence à subárvore esquerda de p.
Além disso, p possui filho esquerdo $u \neq q$.
Sabe-se que $h_E(u) \neq h_D(u)$.



Inclusão em Árvores AVL

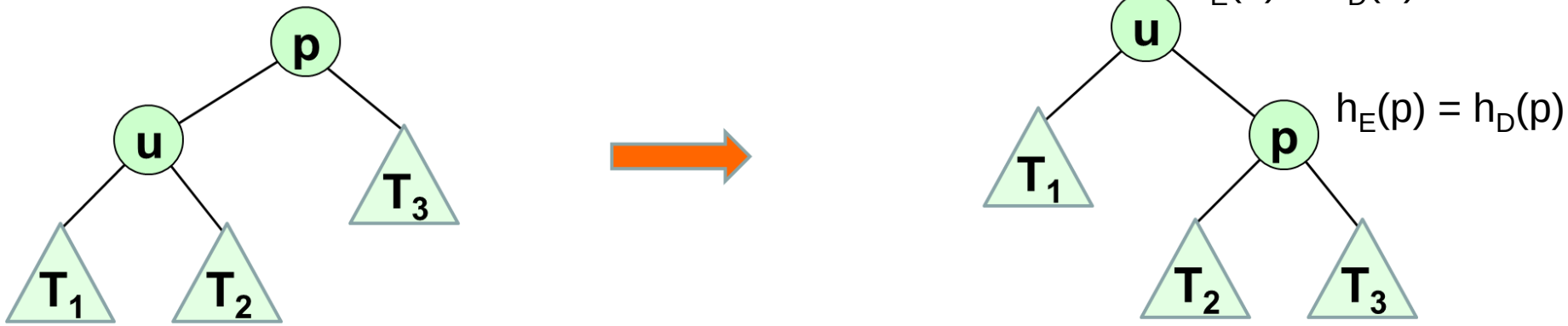
Caso 1.1 $h_E(u) > h_D(u)$

$h(T_1) - h(T_2) = 1$
$h(T_2) = h(T_3)$

Aplicar Rotação Direita

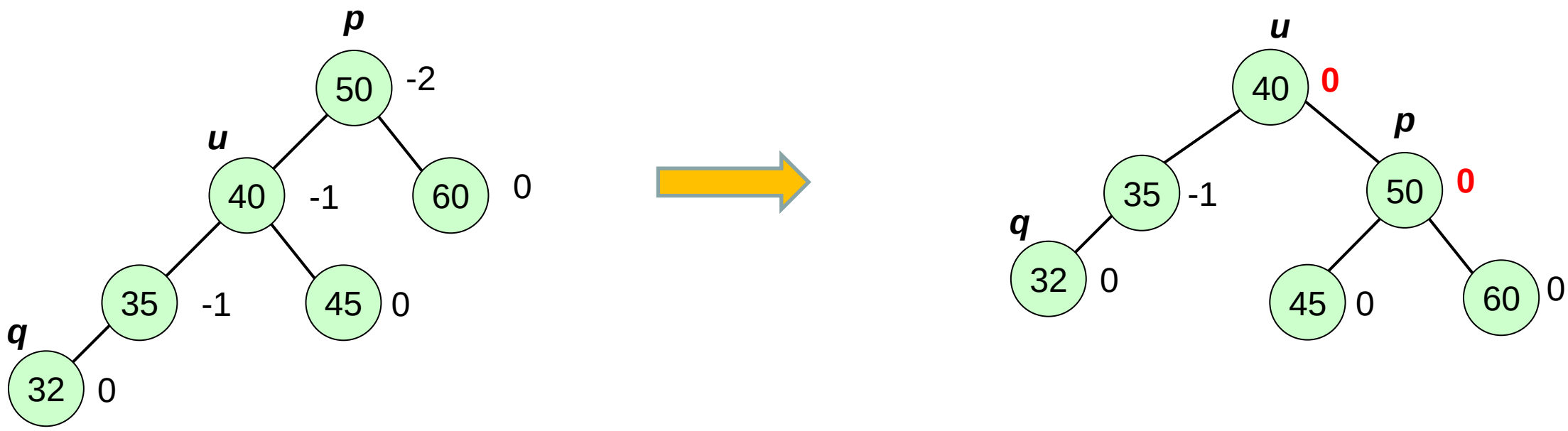
$|h_E(p) - h_D(p)| = 2$
Caso 1. $h_E(p) > h_D(p)$

Neste caso q pertence à subárvore esquerda de p.
Além disso, p possui filho esquerdo $u \neq q$.
Sabe-se que $h_E(u) \neq h_D(u)$.



Inclusão em Árvores AVL

Exemplo Caso 1.1 $h_E(u) > h_D(u)$



Inclusão em Árvores AVL

Caso 1.2 $h_E(u) < h_D(u)$

Então u possui filho direito v.
$|h(T_2) - h(T_3)| \leq 1$

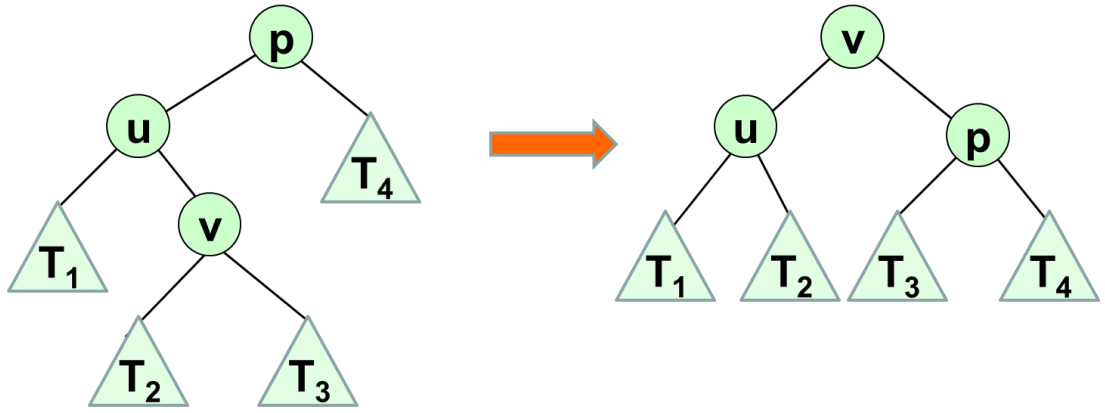
e

$\max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$

Aplicar Rotação Dupla Direita

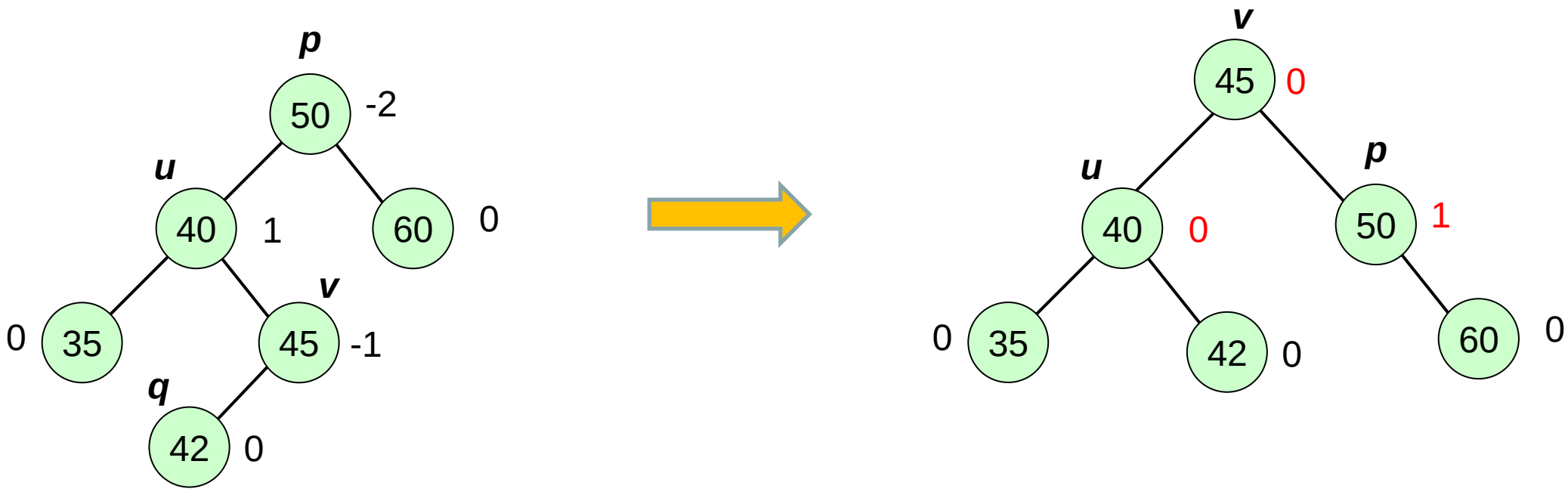
$|h_E(p) - h_D(p)| = 2$
Caso 1. $h_E(p) > h_D(p)$

Neste caso q pertence à subárvore esquerda de p.
Além disso, p possui filho esquerdo $u \neq q$.
Sabe-se que $h_E(u) \neq h_D(u)$.



Inclusão em Árvores AVL

Exemplo Caso 1.2 $h_E(u) < h_D(u)$



Inclusão em Árvores AVL

Quando $|h_E(p) - h_D(p)| = 2$, pode-se identificar as seguintes situações:

Caso 1. $h_E(p) > h_D(p)$

Caso 2. $h_E(p) < h_D(p)$ **– Análogo ao caso 1.**

Inclusão em Árvores AVL

Balanço(v) = $h_D(v)$ - $h_E(v)$

Nó regulado: -1 ≤ balanço(v) ≤ 1

Suponha que o nó q foi inserido na subárvore esquerda de v.

Inclusão em Árvores AVL

Balanço(v) = $h_D(v)$ - $h_E(v)$

Caso 1. balanço(v) = 1 (antes da inclusão)

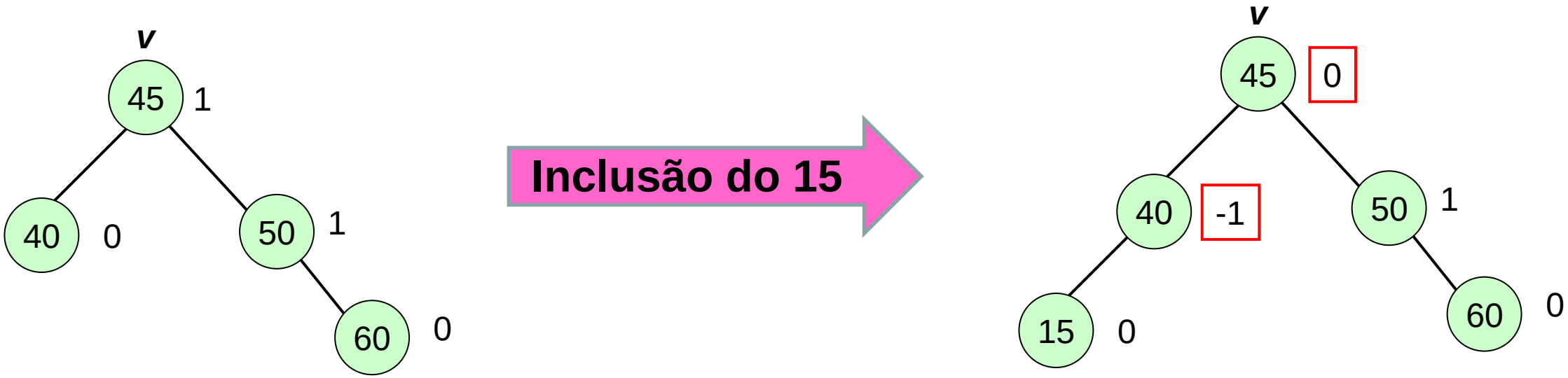
Nesse caso balanço(v) se torna 0 após a inclusão.

A altura de *v* não foi modificada.

Consequentemente, as alturas dos nós entre a raiz e v também não se modificam.

Inclusão em Árvores AVL

Exemplo Caso 1. balanço(v) = 1 (antes da inclusão)



Inclusão em Árvores AVL

$$\text{Balanço}(v) = h_D(v) - h_E(v)$$

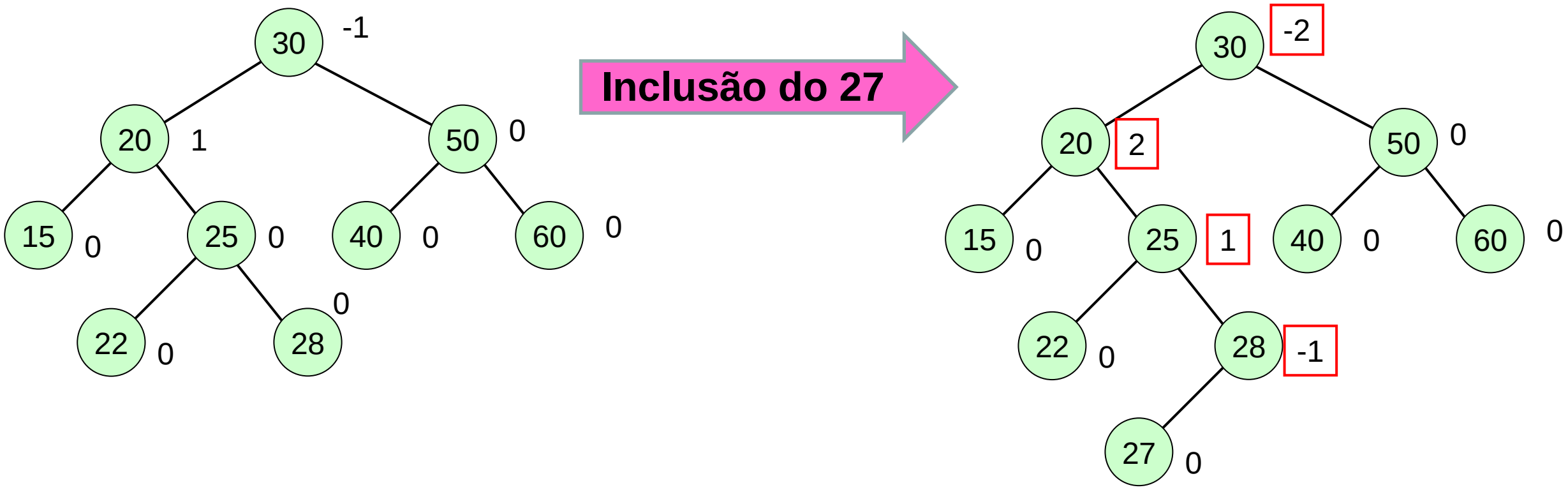
Caso 2. balanço(v) = 0 (antes da inclusão)

Nesse caso balanço(v) torna-se -1.
A altura de *v* foi modificada.
Consequentemente, as alturas dos nós entre a raiz e v podem ter sido modificadas e devem ser analisadas.

Se v é a raiz, então a análise se encerra.
Caso contrário, substituir *v* por seu pai e continuar a análise.

Inclusão em Árvores AVL

Exemplo Caso 2. balanço(v) = 0 (antes da inclusão)



Inclusão em Árvores AVL

$$\text{Balanço}(v) = h_D(v) - h_E(v)$$

Caso 3. balanço(v) = -1 (antes da inclusão)

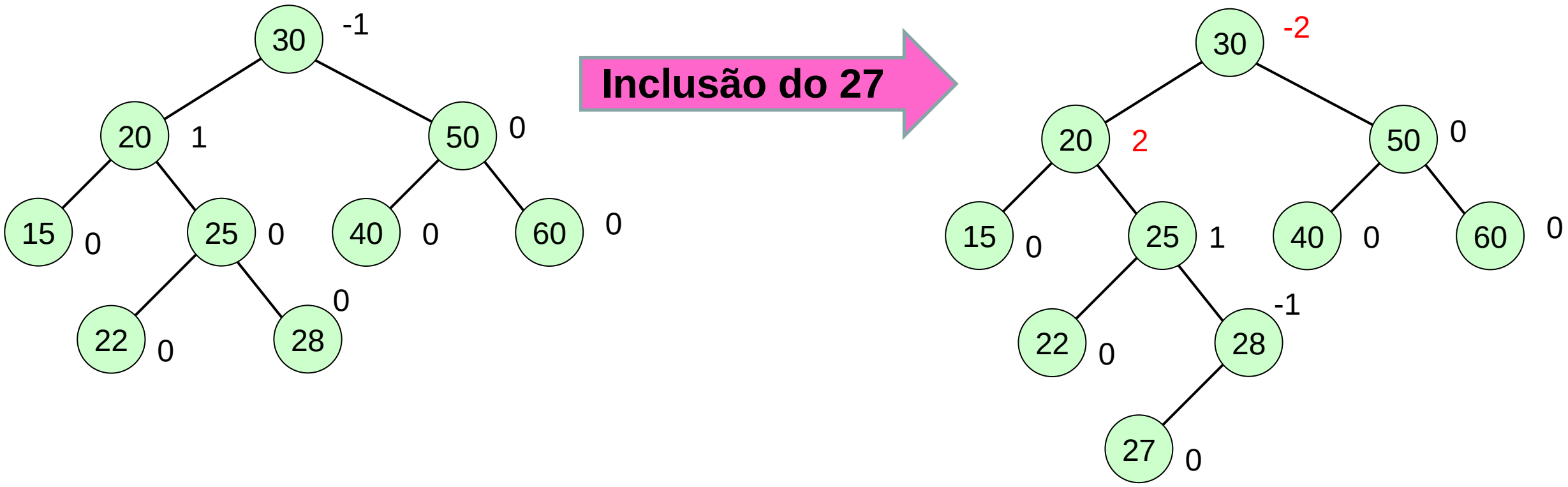
Nesse caso balanço(v) torna-se -2 e o nó está desregulado.

A rotação correta deve ser empregada.

Qualquer rotação implica que a subárvore resultante tenha a mesma altura da subárvore antes da inclusão. As alturas dos ancestrais de v não mais necessitam de avaliação.

Inclusão em Árvores AVL

Exemplo Caso 2. balanço(v) = 0 (antes da inclusão)



Inclusão em Árvore AVL

```
Procedimento cria_no(pt)
    ocupar(pt)
    pt↑.esq ←λ
    pt↑.dir  ←λ
    pt↑.chave ←x
    pt↑.bal ← 0
```

Inclusão em Árvore AVL

```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
       insere_AVL(x, pt↑.esq,h)
       se h então
          caso pt↑.bal seja
          1 : pt↑.bal ← 0; h ← "F"
          0 : pt↑.bal ← -1
          -1: caso1(pt, h)    //Rebalanceamento
     senão
       insere_AVL(x, pt↑.dir,h)
       se h então
          caso pt↑.bal seja
          -1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← 1
           1 : caso2(pt, h)     //Rebalanceamento
```

Inclusão em Árvore AVL

```
Procedimento caso1(pt,h)
    ptu ← pt↑.esq
    se ptu↑.bal = -1 então
          pt↑.esq ← ptu↑.dir; ptu↑.dir ← pt
          pt↑.bal ← 0; pt ← ptu
    senão
          ptv ←  ptu↑.dir
          ptu↑.dir ← ptv↑.esq ; ptv↑.esq ← ptu
          pt↑.esq ← ptv↑.dir ; ptv↑.dir ← pt
          se (ptv↑.bal = -1) então pt↑.bal ← 1 senão pt↑.bal ← 0
          se (ptv↑.bal =  1) então ptu↑.bal ← -1 senão ptu↑.bal ← 0
          pt ←  ptv
    pt↑.bal ← 0 ; h ← "F"
```

Rotação Direita

Rot Dup Dir

Inclusão em Árvore AVL

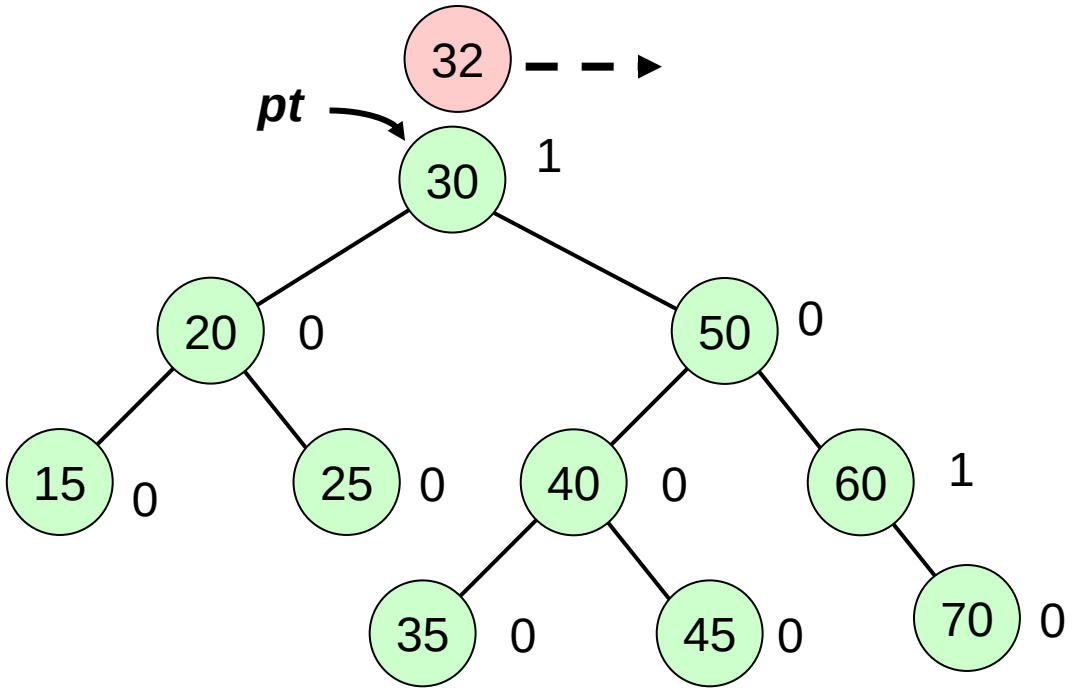
```
Procedimento caso2(pt,h)
    ptu ← pt↑.dir
    se ptu↑.bal = 1 então
          pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
          pt↑.bal ← 0; pt ← ptu
    senão
          ptv ←  ptu↑.esq
          ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
          pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
          se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
          se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
          pt ←  ptv
    pt↑.bal ← 0 ; h ← "F"
```

Rotação Esquerda

Rot Dup Esq

Exemplo: Inclusão em Árvore AVL

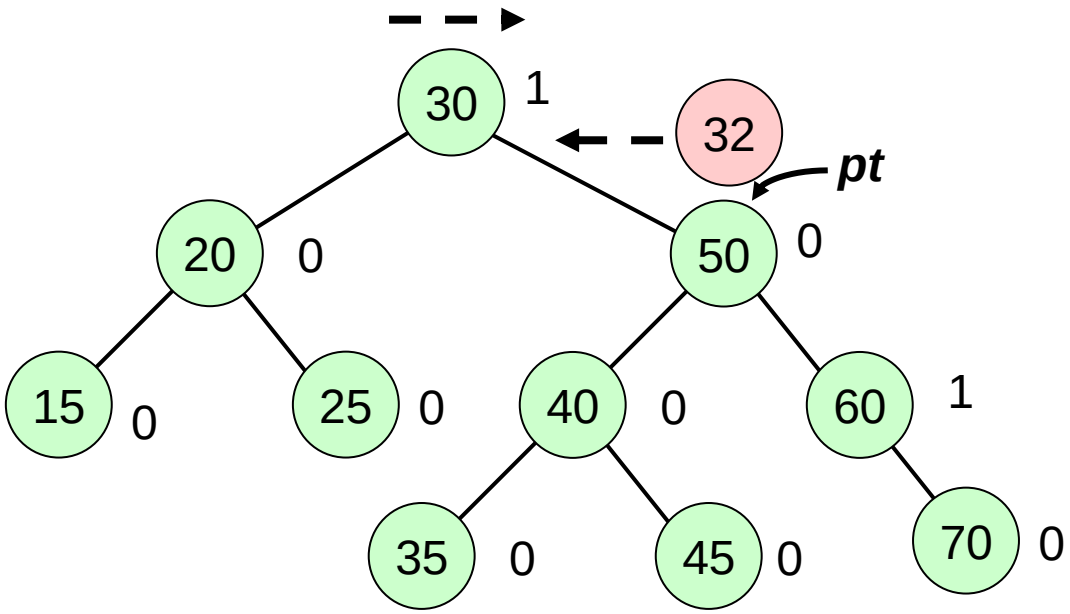
Inclusão da chave 32




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

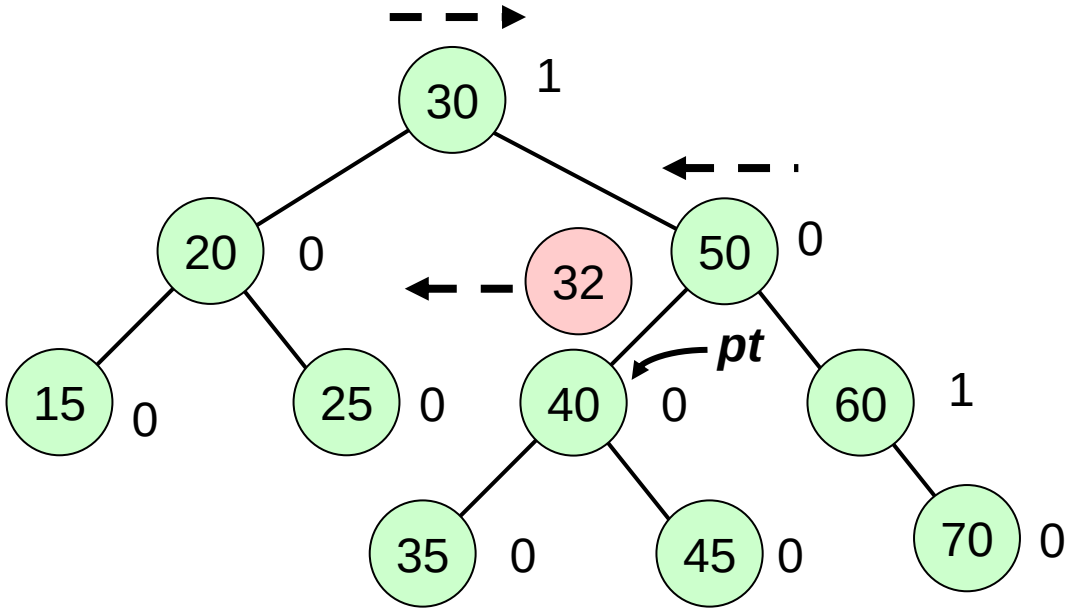
Inclusão da chave 32




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
       insere_AVL(x, pt↑.esq,h)
       se h então
          caso pt↑.bal seja
          1 : pt↑.bal ← 0; h ← "F"
          0 : pt↑.bal ← -1
          -1: caso1(pt, h)    //Rebalanceamento
     senão
       insere_AVL(x, pt↑.dir,h)
       se h então
          caso pt↑.bal seja
          -1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← 1
           1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

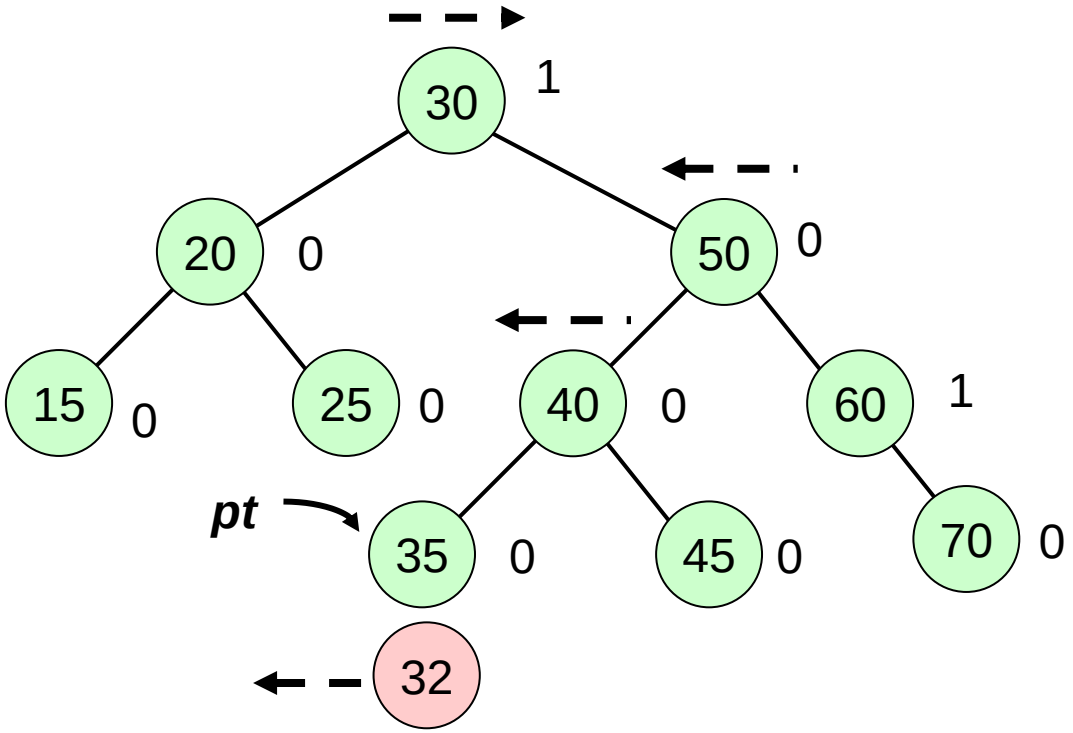
Inclusão da chave 32




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
       insere_AVL(x, pt↑.esq,h)
       se h então
          caso pt↑.bal seja
          1 : pt↑.bal ← 0; h ← "F"
          0 : pt↑.bal ← -1
          -1: caso1(pt, h)    //Rebalanceamento
     senão
       insere_AVL(x, pt↑.dir,h)
       se h então
          caso pt↑.bal seja
          -1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← 1
           1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

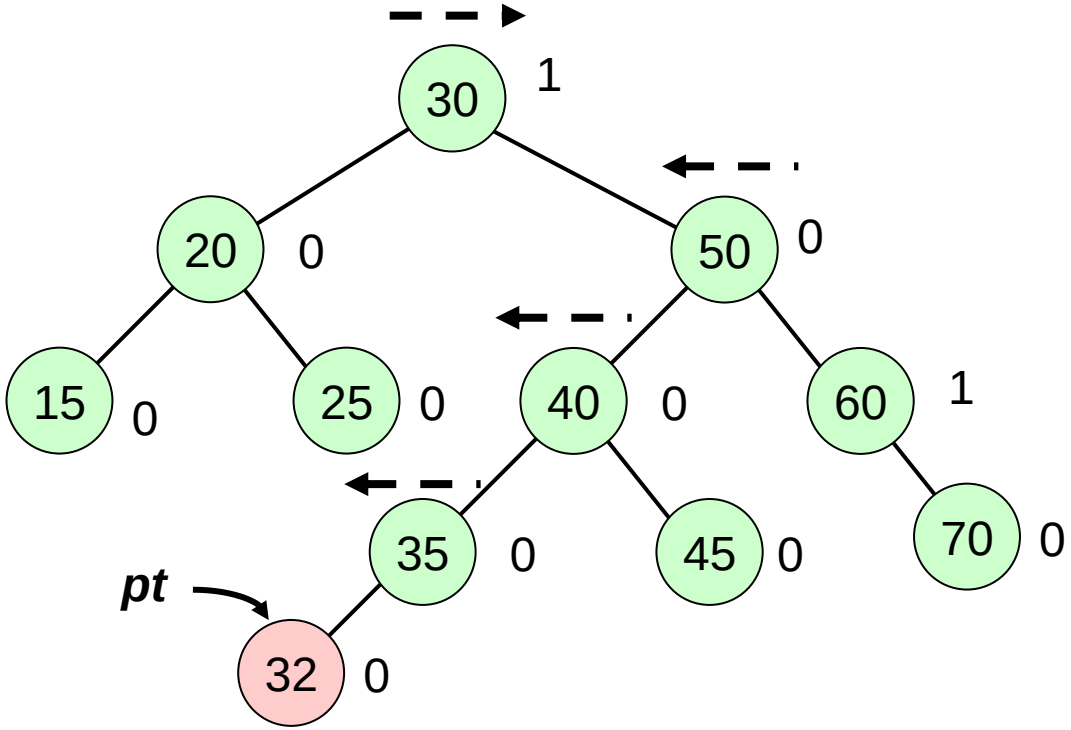
Inclusão da chave 32




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
       insere_AVL(x, pt↑.esq,h)
       se h então
          caso pt↑.bal seja
          1 : pt↑.bal ← 0; h ← "F"
          0 : pt↑.bal ← -1
          -1: caso1(pt, h)    //Rebalanceamento
     senão
       insere_AVL(x, pt↑.dir,h)
       se h então
          caso pt↑.bal seja
          -1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← 1
           1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

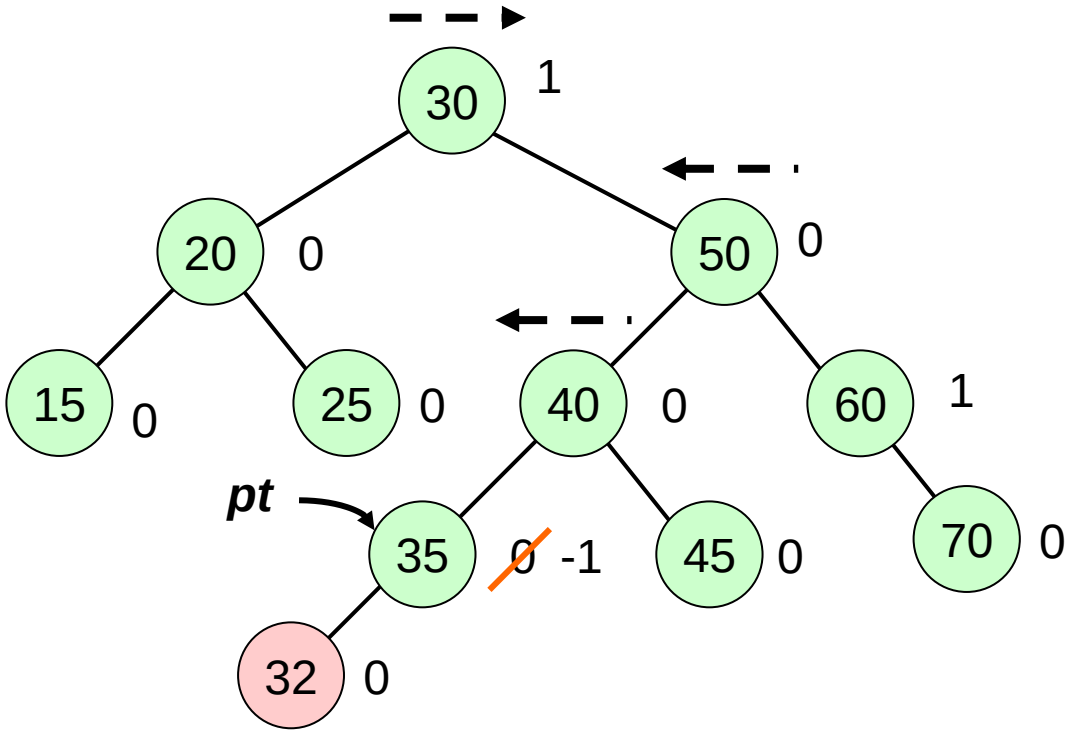
Inclusão da chave 32




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

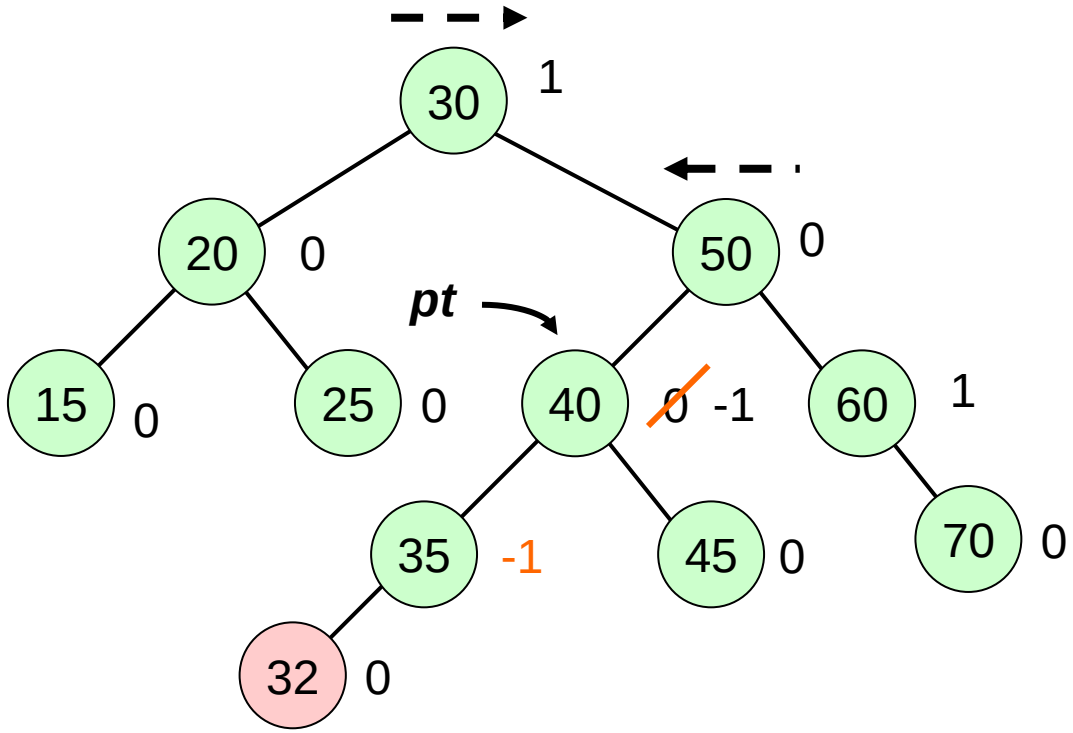
Volta do Algoritmo Recalculando o Balanço




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

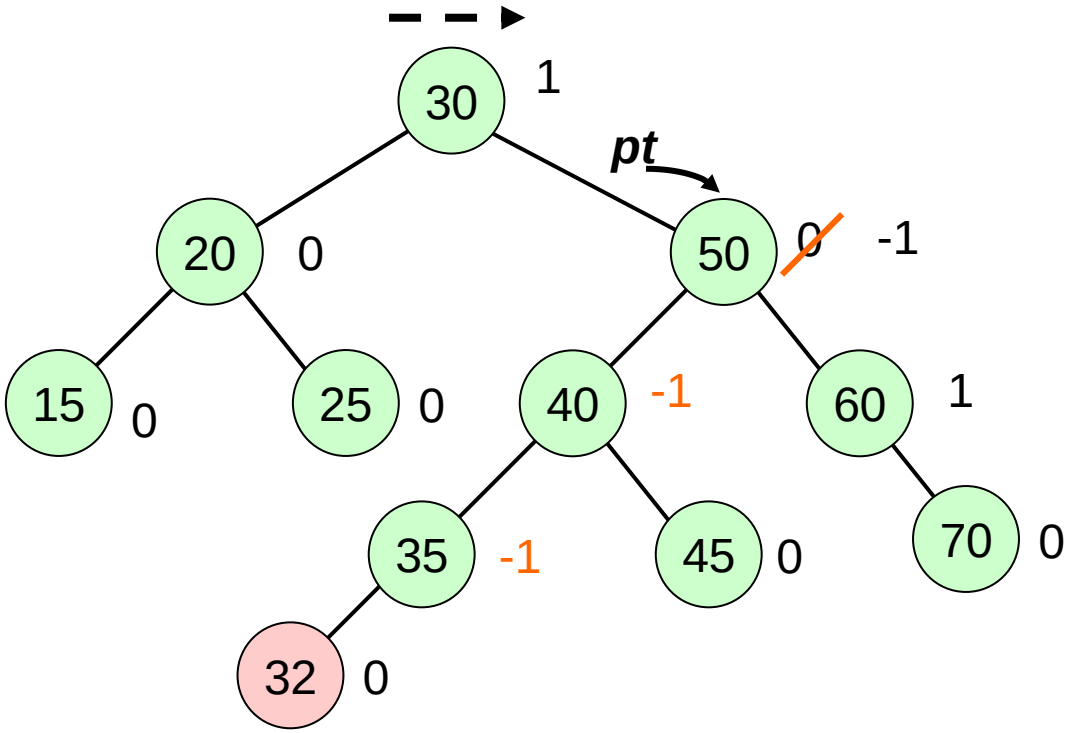
Volta do Algoritmo Recalculando o Balanço




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

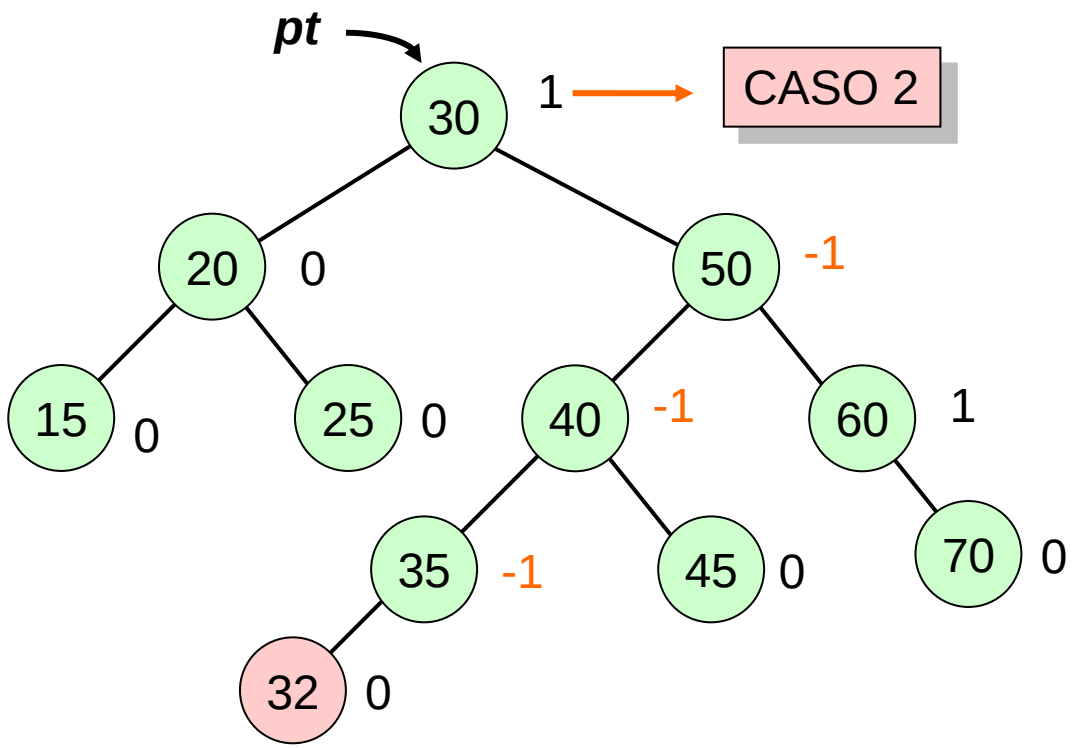
Volta do Algoritmo Recalculando o Balanço




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

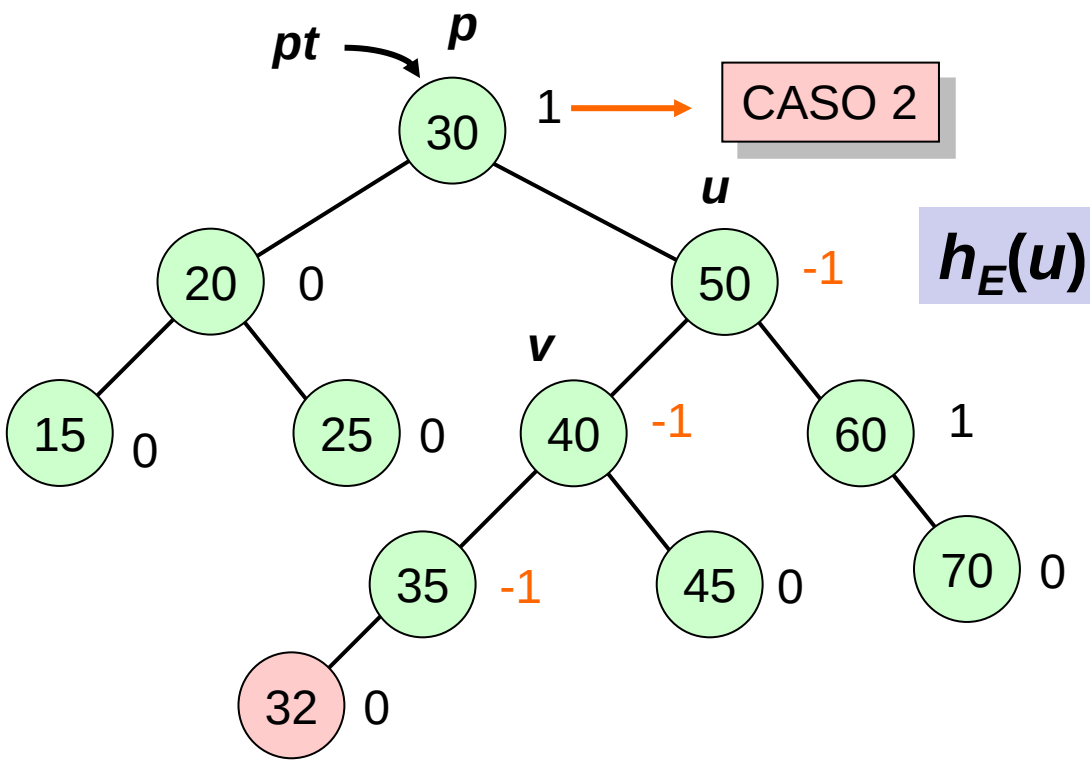
Volta do Algoritmo Recalculando o Balanço




```
Procedimento insere_AVL(x, pt, h)
  se pt = λ então cria_no(pt); h ← "V"
  senão
     se (x = pt↑.chave) então PARE
     se (x < pt↑.chave) então
        insere_AVL(x, pt↑.esq,h)
        se h então
           caso pt↑.bal seja
           1 : pt↑.bal ← 0; h ← "F"
           0 : pt↑.bal ← -1
           -1: caso1(pt, h)    //Rebalanceamento
     senão
        insere_AVL(x, pt↑.dir,h)
        se h então
           caso pt↑.bal seja
           -1 : pt↑.bal ← 0; h ← "F"
            0 : pt↑.bal ← 1
            1 : caso2(pt, h)    //Rebalanceamento
```

Inclusão em Árvore AVL

Volta do Algoritmo Recalculando o Balanço



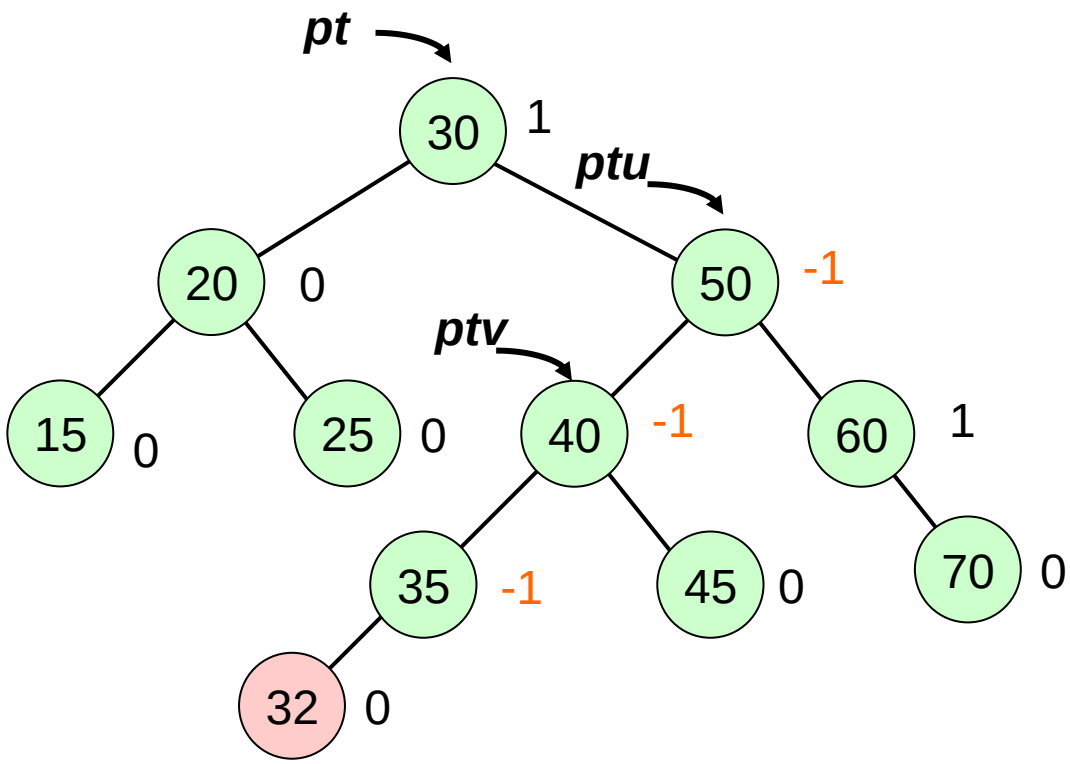
$h_E(u) > h_D(u)$

Rotação dupla esquerda com raiz p

Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

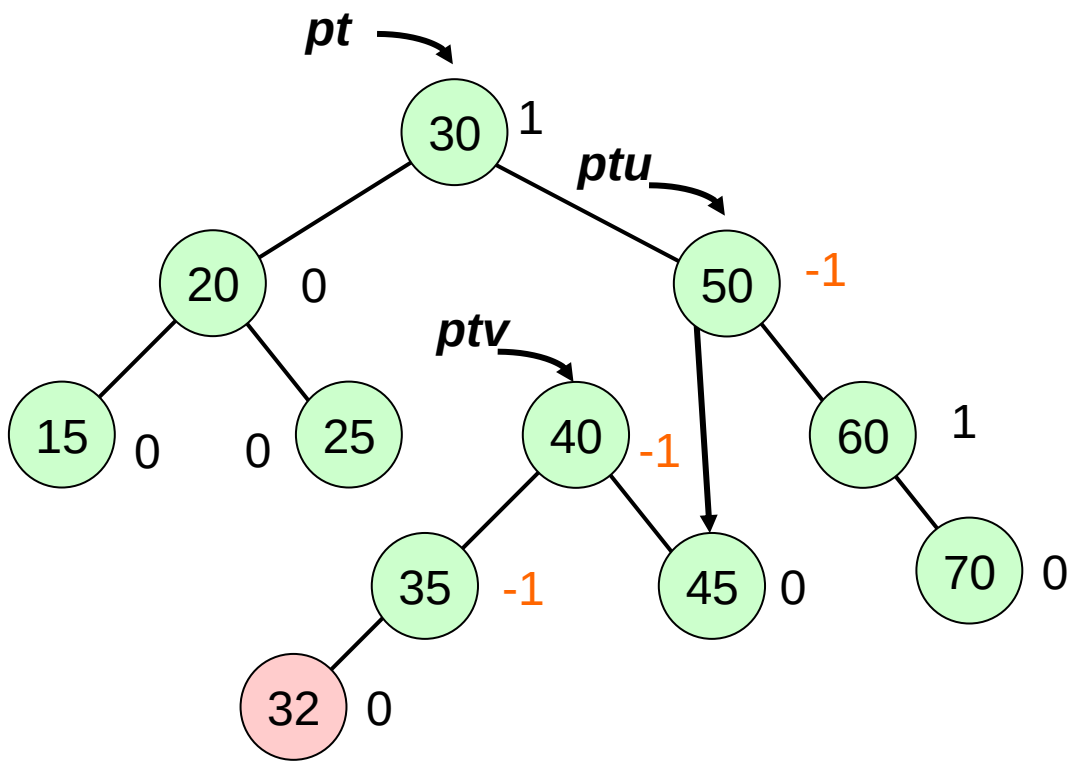
```
Procedimento caso2(pt,h)
   ptu ← pt↑.dir
   se ptu↑.bal = 1 então
        pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
        pt↑.bal ← 0; pt ← ptu
   senão
        ptv ←  ptu↑.esq
        ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
        pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
        se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
        se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
        pt ←  ptv
   pt↑.bal ← 0 ; h ← "F"
```



Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

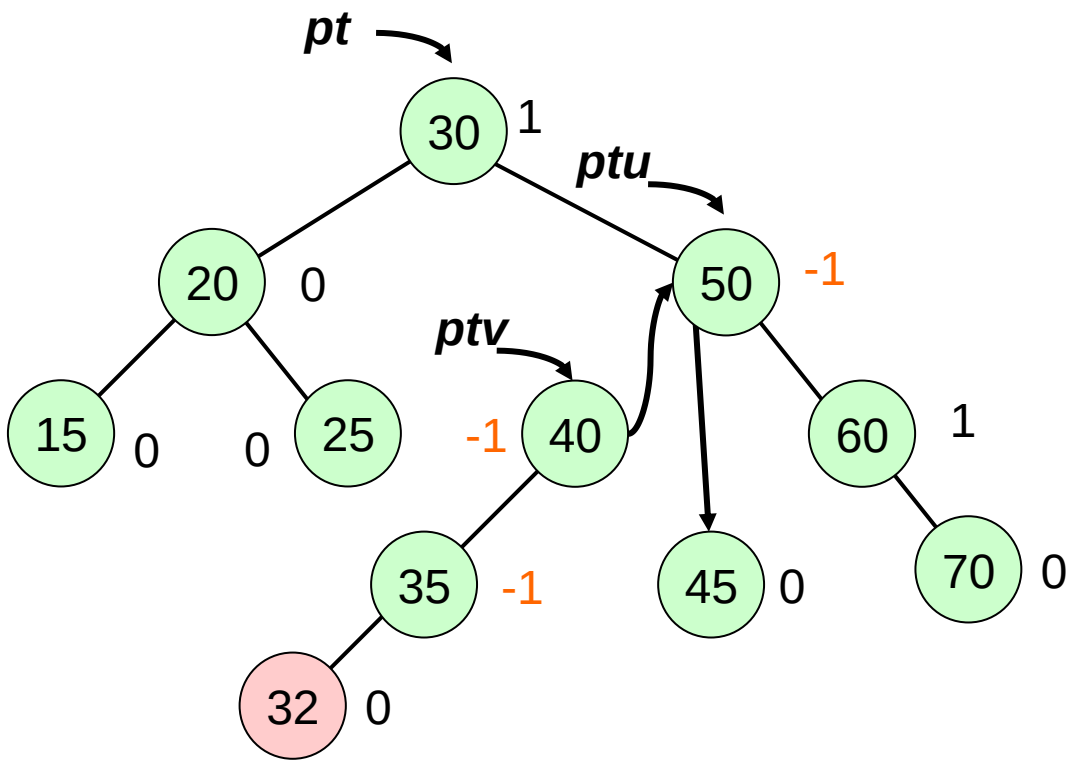
```
Procedimento caso2(pt,h)
    ptu ← pt↑.dir
    se ptu↑.bal = 1 então
         pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
         pt↑.bal ← 0; pt ← ptu
    senão
         ptv ←  ptu↑.esq
         ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
         pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
         se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
         se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
         pt ←  ptv
    pt↑.bal ← 0 ; h ← "F"
```



Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

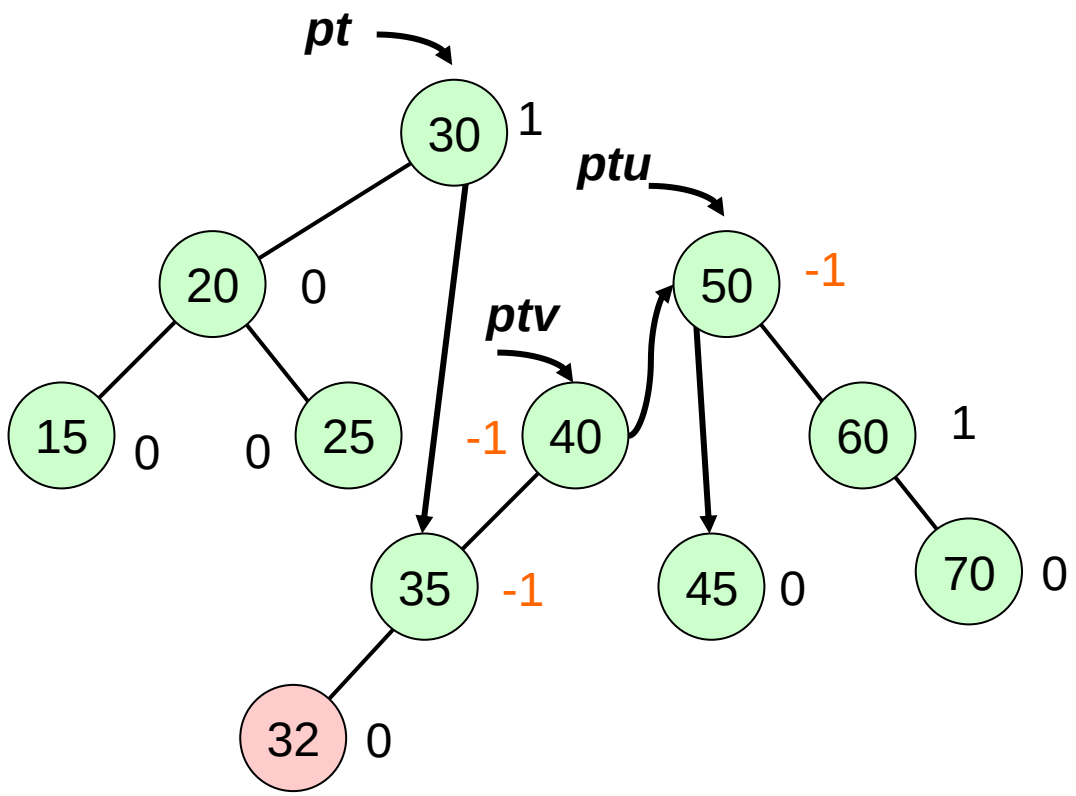
```
Procedimento caso2(pt,h)
    ptu ← pt↑.dir
    se ptu↑.bal = 1 então
         pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
         pt↑.bal ← 0; pt ← ptu
    senão
         ptv ←  ptu↑.esq
         ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
         pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
         se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
         se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
         pt ←  ptv
    pt↑.bal ← 0 ; h ← "F"
```



Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

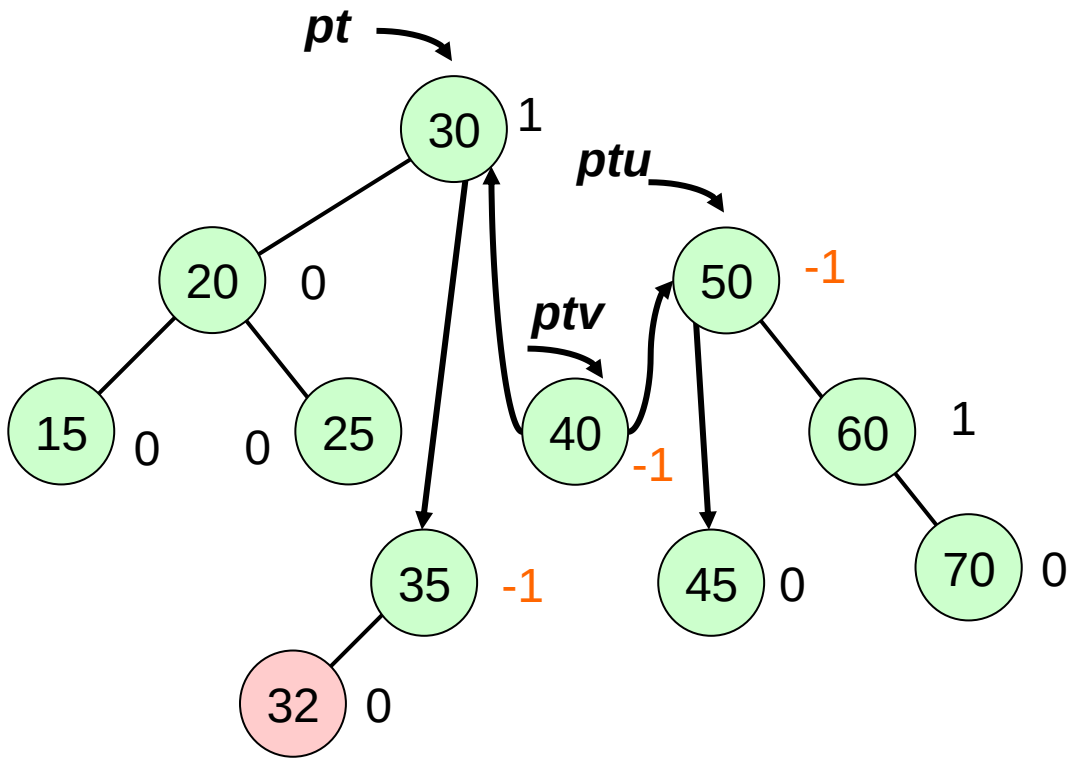
```
Procedimento caso2(pt,h)
   ptu ← pt↑.dir
   se ptu↑.bal = 1 então
        pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
        pt↑.bal ← 0; pt ← ptu
   senão
        ptv ←  ptu↑.esq
        ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
        pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
        se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
        se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
        pt ←  ptv
   pt↑.bal ← 0 ; h ← "F"
```



Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

```
Procedimento caso2(pt,h)
   ptu ← pt↑.dir
   se ptu↑.bal = 1 então
        pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
        pt↑.bal ← 0; pt ← ptu
   senão
        ptv ←  ptu↑.esq
        ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
        pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
        se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
        se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
        pt ←  ptv
   pt↑.bal ← 0 ; h ← "F"
```

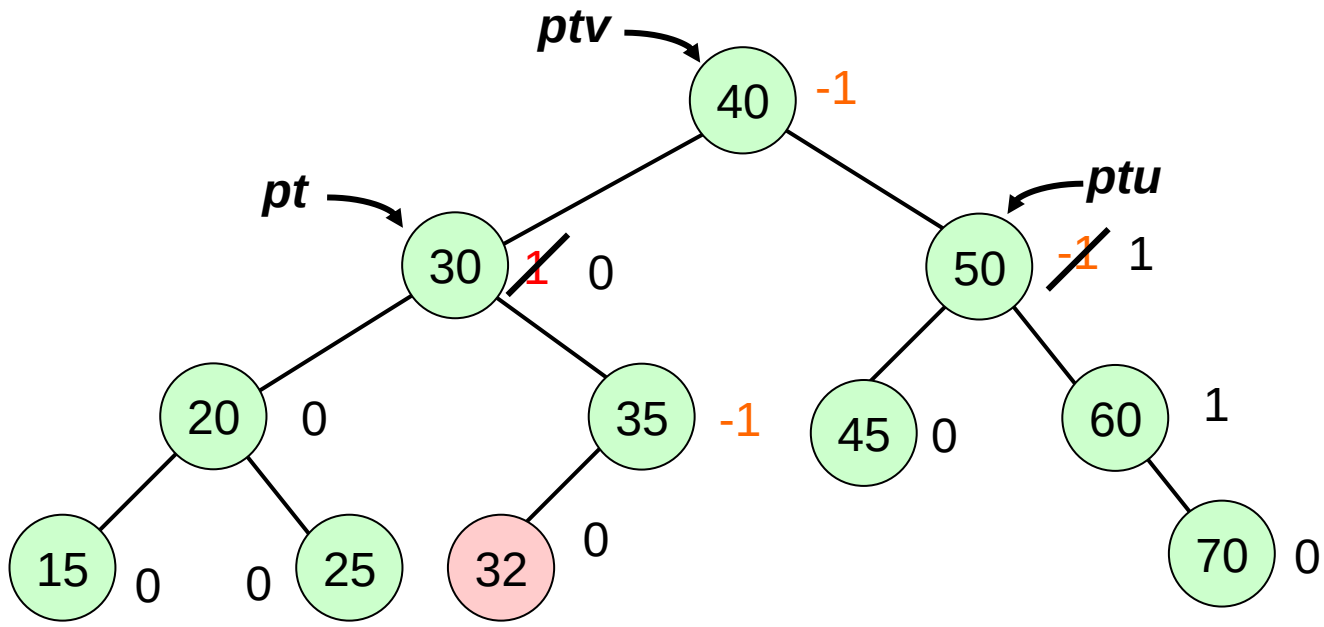


Redesenhando!!!

Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

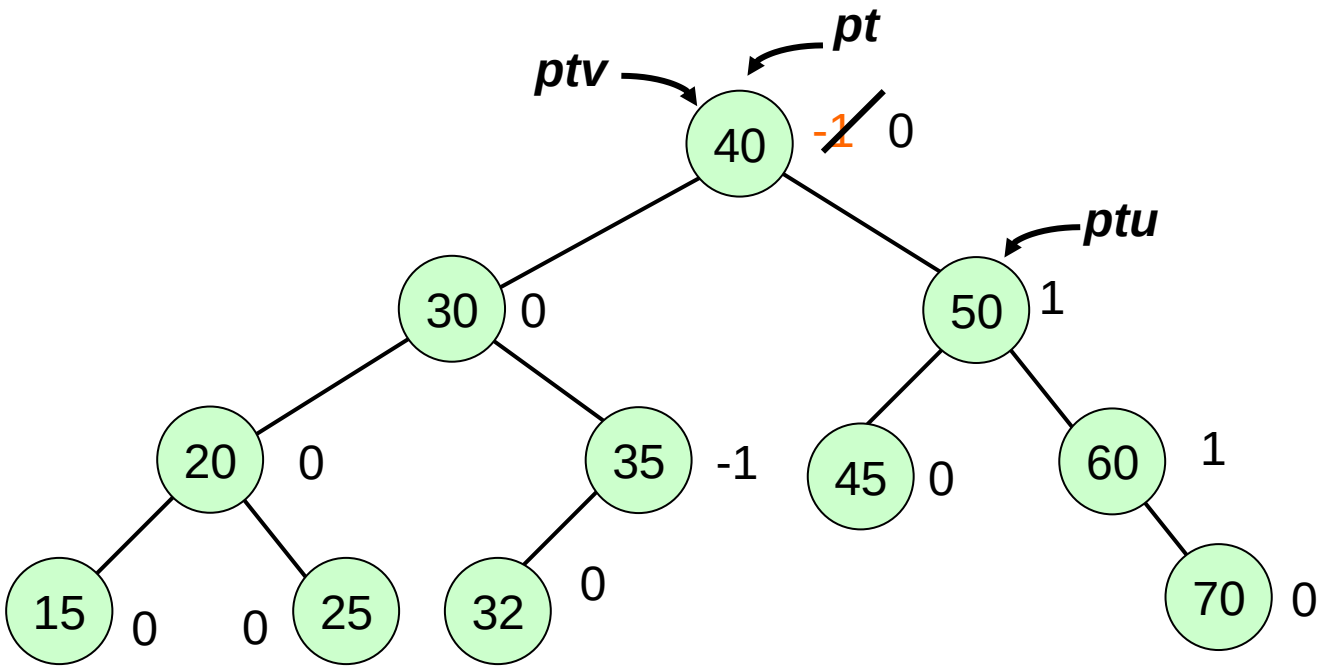
```
Procedimento caso2(pt,h)
   ptu ← pt↑.dir
   se ptu↑.bal = 1 então
        pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
        pt↑.bal ← 0; pt ← ptu
   senão
        ptv ←  ptu↑.esq
        ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
        pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
        se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
        se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
        pt ←  ptv
   pt↑.bal ← 0 ; h ← "F"
```



Inclusão em Árvore AVL

Balanceamento: Rotação Dupla Esquerda

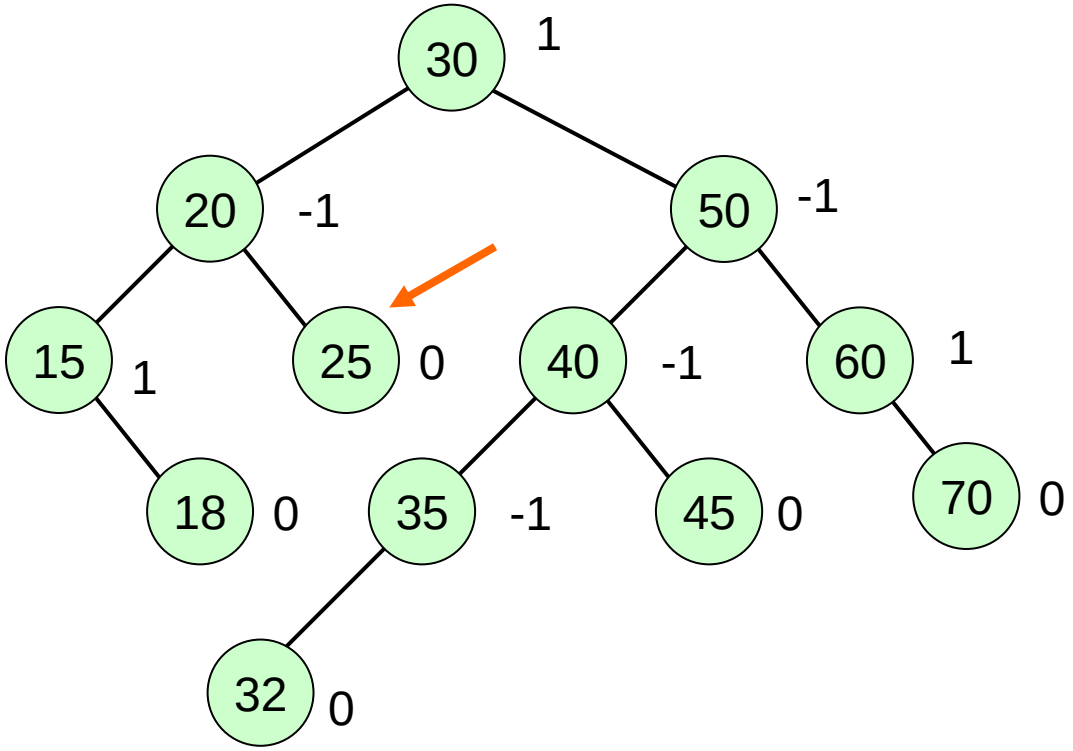
```
Procedimento caso2(pt,h)
   ptu ← pt↑.dir
   se ptu↑.bal = 1 então
        pt↑.dir ← ptu↑.esq; ptu↑.esq ← pt
        pt↑.bal ← 0; pt ← ptu
   senão
        ptv ←  ptu↑.esq
        ptu↑.esq ← ptv↑.dir ; ptv↑.dir ← ptu
        pt↑.dir← ptv↑.esq ; ptv↑.esq ← pt
        se (ptv↑.bal =  1) então pt↑.bal ← -1 senão pt↑.bal ← 0
        se (ptv↑.bal = -1) então ptu↑.bal ← 1 senão ptu↑.bal ← 0
        pt ←  ptv
   pt↑.bal ← 0 ; h ← "F"
```



h ←"F"

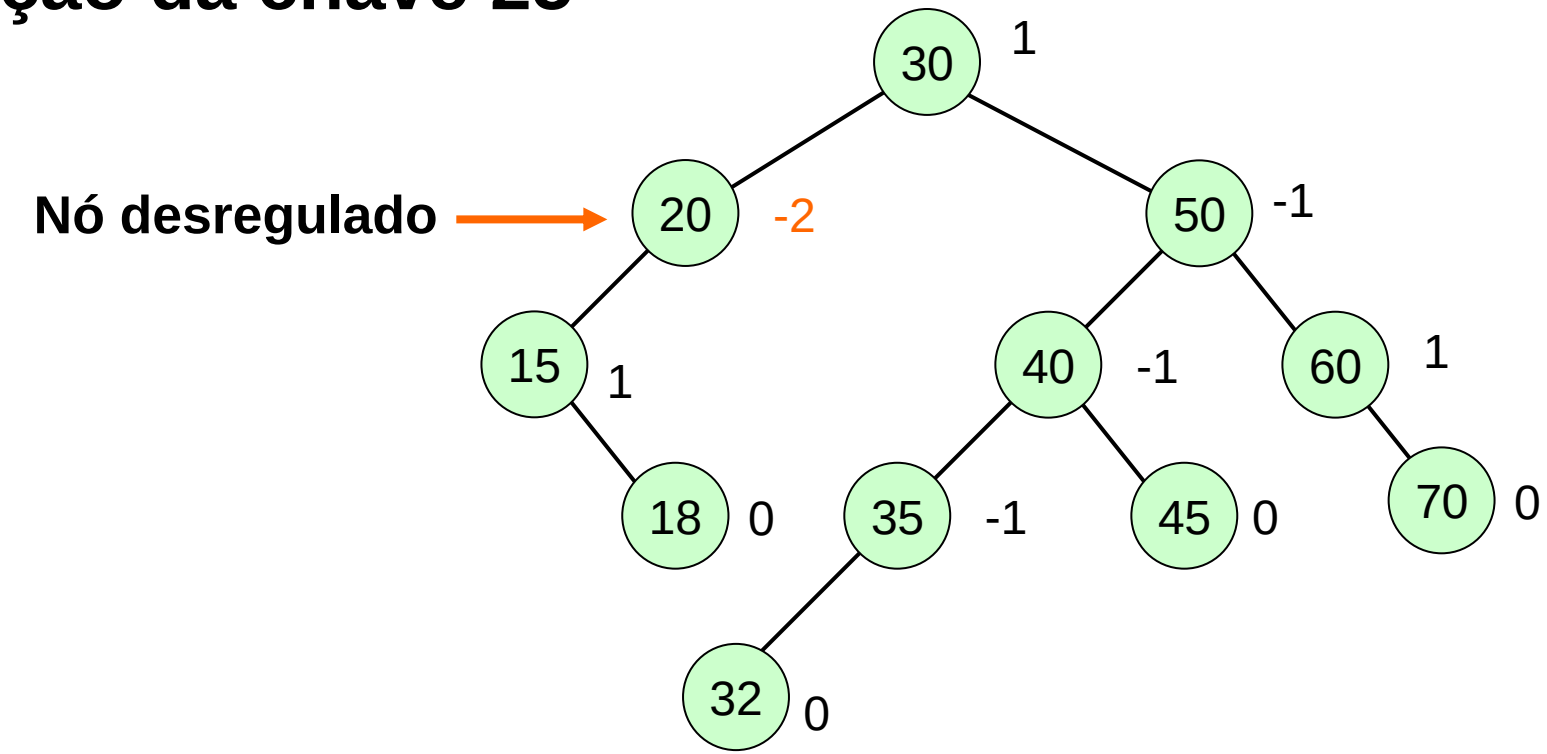
Remoção em Árvore AVL

Remoção da chave 25



Remoção em Árvore AVL

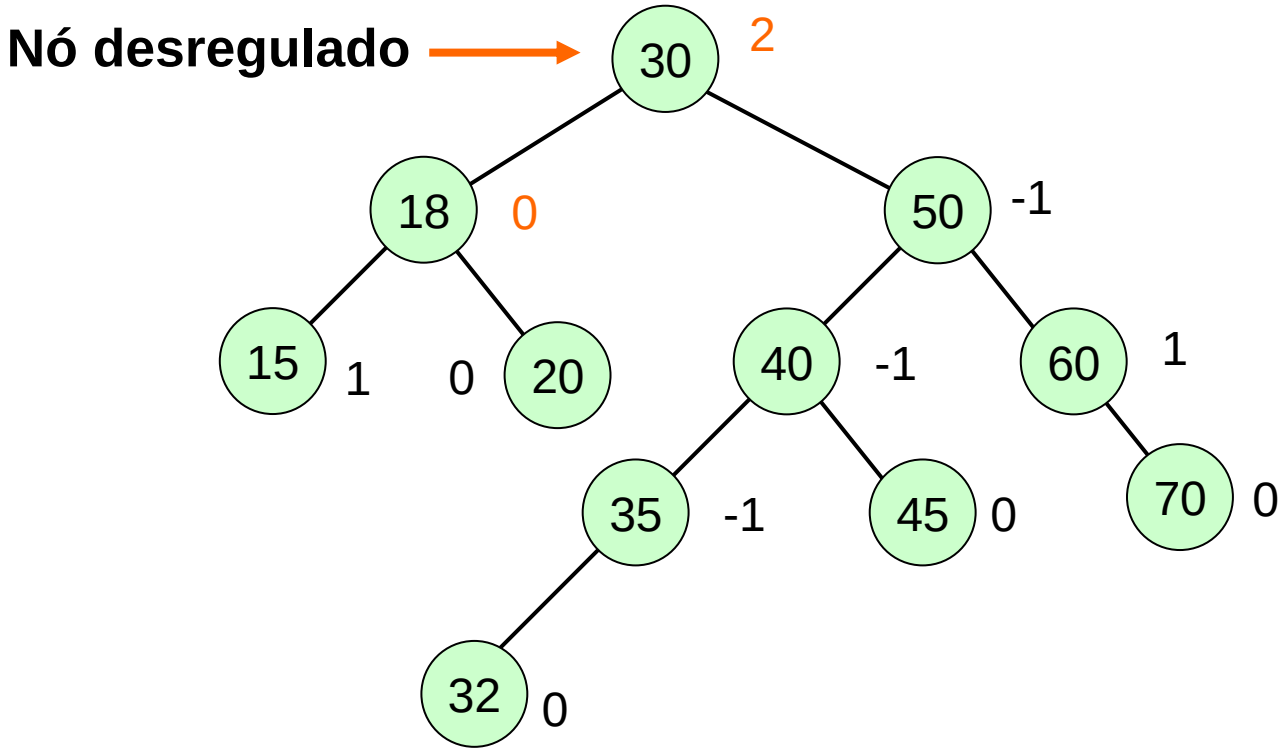
Remoção da chave 25



Aplicar: Rotação Dupla Direita

Remoção em Árvore AVL

Remoção da chave 25



Aplicar: Rotação Dupla Esquerda

Remoção em Árvore AVL

Remoção da chave 25

